

Pełna dokumentacja v3

Mikrobenchmarki → walidacja FEM → referencja exact → replay poprawności → korelacja profilerowa

Zakres dokumentu

- Wprowadzenie do v3
- Teoria od podstaw
- Rozdział 1. Motywacja i kontekst badań
- Rozdział 2. Obliczenia numeryczne i FEM
- Rozdział 3. Architektura i backendy
- Rozdział 4. Metodologia badawcza i walidacja
- Rozdział 5. Metryki, profiling i interpretacja
- Rozdział 6. Synteza wkładu i zakres wniosków
- Przewodnik startowy
- Indeks dokumentacji
- Mapa projektu
- Referencja systemowa
- Metodologia: od mikrobenchmarków do FEM
- Wykresy czasu wykonania i autotuningu
- Plan badawczy rozprawy
- Mapa nazewnictwa do rozprawy
- Protokół eksperymentalny
- Zagrożenia dla trafności
- Przepływ pracy od testów do pisania
- Konfiguracja Linux/OpenCL i exact
- Schemat danych CSV
- Definicje metryk
- Odtwarzalność eksperymentów

Wygenerowano: 2026-04-19T12:11:34.697494+02:00

Katalog roboczy: .../v3

Spis treści

Sekcje ułożone w kolejności czytania i pracy eksperymentalnej.

1. Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

2. Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

3. Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

4. Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

5. Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

6. Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

7. Rozdział 5. Metryki, profiling i interpretacja

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

8. Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

9. Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

10. Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

Pełna dokumentacja v3

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

Plik źródłowy: README.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

v3 to uporządkowana wersja platformy badawczej, w której:

- mikrobenchmarki są główną warstwą badawczą,
- Walidacja opcji FEM jest warstwą interpretacyjną bliską realistycznemu kernelowi FEM,
- referencyjny exact dostarcza zamrożone dane wejściowe i oczekiwane wyniki,
- replay poprawności pozwala porównywać backendy na tych samych danych,
- korelacja profilerowa łączy wyniki architektoniczne z zachowaniem realistycznego kernela.

Szybki start

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python run_autotune_gui.py
```

```
```
```

Najważniejsze PDF-y

- docs/v3_teorja_od_podstaw.pdf
- docs/v3_ready_reference.pdf
- docs/v3_documentation_bundle.pdf

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

- docs/v3_rozdzialy_teoretyczne.pdf

Budowanie PDF-ow:

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python scripts/build_v3_documentation_pdf.py
```

```
```
```

Od czego zaczac

Jesli chcesz uruchamiac eksperymenty

1. docs/V3_READY_REFERENCE.md
2. docs/PROJECT_MAP.md
3. docs/SYSTEM_REFERENCE.md
4. docs/END_TO_END_TESTING_AND_WRITING.md

Jesli chcesz pisac rozprawe

1. docs/V3_DOCUMENTATION_INDEX.md

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

2. docs/TEORIA_OD_PODSTAW.md
3. docs/V3_READY_REFERENCE.md
4. docs/THESIS_RESEARCH_PLAN.md
5. docs/METHODOLOGY_MICROBENCH_TO_FEM.md
6. docs/EXPERIMENTAL_PROTOCOL.md
7. docs/THREATS_TO_VALIDITY.md

Najwazniejsze przepływy pracy

- cpu_benchmark
- gpu_benchmark
- filip_original
- fem_option_validation
- profiler_correlation
- filip_autotune
- filip_firefly

Najwazniejsze dokumenty

- docs/TEORIA_OD_PODSTAW.md

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

- docs/ROZDZIAL_01_MOTYWACJA_I_KONTEKST.md
- docs/ROZDZIAL_02_OBLICZENIA_NUMERYCZNE_I_FEM.md
- docs/ROZDZIAL_03_ARCHITEKTURA_I_BACKENDY.md
- docs/ROZDZIAL_04_METODOLOGIA_BADAWCZA_I_WALIDACJA.md
- docs/ROZDZIAL_05_METRYKI_PROFILING_I_INTERPRETACJA.md
- docs/ROZDZIAL_06_SYNTEZA_WKLADU_I_ZAKRES_WNIOSKOW.md
- docs/V3_DOCUMENTATION_INDEX.md
- docs/V3_READY_REFERENCE.md
- docs/PROJECT_MAP.md
- docs/SYSTEM_REFERENCE.md
- docs/METHODOLOGY_MICROBENCH_TO_FEM.md
- docs/THESIS_RESEARCH_PLAN.md
- docs/EXPERIMENTAL_PROTOCOL.md
- docs/THREATS_TO_VALIDITY.md
- docs/END_TO_END_TESTING_AND_WRITING.md

Jednozdaniowa narracja v3

- v3 łączy mikrobenchmarki architektury z realistycznym kernelem FEM przez warstwy próbek walidacyjnych, replay poprawności na zamrożonych danych wejściowych oraz korelacje profilowe.

Wprowadzenie do v3

Krótki opis platformy i punkt startowy do pracy.

Jesli zaczynasz od zera

Najbezpieczniejsza kolejnosc dla osoby spoza informatyki jest taka:

1. docs/TEORIA_OD_PODSTAW.md
2. docs/v3_teorja_od_podstaw.pdf
3. docs/v3_rozdzialy_teoretyczne.pdf
4. docs/V3_READY_REFERENCE.md

Pełna dokumentacja v3

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Plik źródłowy: docs/TEORIA_OD_PODSTAW.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Ten dokument jest napisany dla osoby, która:

- nie jest informatykiem,
- nie programuje zawodowo,
- nie zajmuje się architekturą komputerów,
- ale chce dobrze zrozumieć, co jest badane w v3, po co ten system powstał i dlaczego ma sens naukowy.

To nie jest instrukcja techniczna. To jest spokojne wprowadzenie do idei projektu.

Można go czytać jak mapę pojęć: od najprostszych intuicji do bardziej specjalistycznych warstw eksperymentu.

1. Po co w ogóle istnieje ten projekt

Najprostsza odpowiedź brzmi:

- chcemy zrozumieć, dlaczego to samo obliczenie może działać szybciej albo wolniej na różnych komputerach,
- ale jednocześnie chcemy mieć pewność, że porównujemy to samo obliczenie, a nie dwa podobne, lecz jednak różne programy.

To bardzo ważne, bo w badaniach wydajności łatwo popełnić błąd interpretacyjny.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Można zobaczyć, że:

- jeden system jest szybszy,
- drugi jest wolniejszy,
- wykres wygląda przekonująco,

ale w rzeczywistości:

- dane wejściowe były inne,
- wynik końcowy nie był sprawdzony,
- albo program wykonywał trochę inną sekwencję obliczeń.

W takim przypadku szybki wynik nie musi jeszcze oznaczać, że porównanie jest uczciwe.

Dlatego v3 nie bada tylko czasu wykonania.

Bada trzy rzeczy jednocześnie:

1. wydajność - jak szybko system wykonuje pracę,
2. poprawność - czy wynik obliczeń jest zgodny z referencją,
3. interpretację - dlaczego uzyskano taki wynik i co on mówi o architekturze sprzętu.

To właśnie odróżnia platformę badawczą od zwykłego zestawu testów wydajnościowych.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

2. Czym w ogóle jest obliczenie numeryczne

W naukach technicznych bardzo często chcemy odpowiedzieć na pytania typu:

- jak rozchodzi się ciepło,
- jak płynie ciecz,
- jak odkształca się materiał,
- jak zmienia się pole w pewnym obszarze,
- jak zachowuje się układ fizyczny w czasie.

Takie zjawiska opisuje się zwykle matematyką, najczęściej w postaci równań różniczkowych.

W teorii brzmi to elegancko, ale w praktyce bardzo rzadko da się rozwiązać taki problem dokładnie "na kartce" dla rzeczywistej geometrii i rzeczywistych warunków.

Wtedy wchodzi obliczenie numeryczne.

To znaczy:

- zamiast próbować znaleźć idealne rozwiązanie wzorem,
- budujemy przybliżenie,
- dzielimy problem na mniejsze części,
- i pozwalamy komputerowi policzyć wynik krok po kroku.

Można to porównać do mierzenia bardzo skomplikowanej linii brzegowej.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Nie da się jej jednym ruchem opisać idealnie, więc dzielimy ją na małe odcinki i sumujemy przybliżenia.

W obliczeniach numerycznych robimy coś podobnego, tylko na dużo większą skalę i w dużo bardziej złożonych problemach.

3. Model, symulacja i wynik - trzy różne rzeczy

Warto od razu odróżnić trzy pojęcia.

3.1. Model

Model to opis badanego zjawiska.

Na przykład:

- model przewodzenia ciepła,
- model przepływu,
- model odkształcenia.

Model mówi, co opisujemy.

3.2. Symulacja

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Symulacja to wykonanie obliczenia tego modelu na komputerze.

Symulacja mówi, jak komputer próbuje policzyć model.

3.3. Wynik

Wynik to konkretna liczba, pole, macierz, wykres lub zbiór danych otrzymany po symulacji.

Wynik odpowiada na pytanie: co wyszło.

To rozróżnienie jest ważne, bo w v3 nie badamy samej fizyki modelu.

Badamy także:

- jak zorganizowano obliczenie,
- jak sprzęt radzi sobie z tym obliczeniem,
- i czy wynik końcowy pozostał zgodny mimo zmiany backendu lub architektury.

4. Czym jest metoda elementów skończonych (FEM)

Jedną z najważniejszych metod stosowanych w obliczeniach numerycznych jest metoda elementów skończonych, czyli FEM.

Jej intuicja jest prosta.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Zamiast patrzeć na cały obszar jako na jeden ogromny i trudny problem, dzieli się go na wiele małych fragmentów.

Te fragmenty nazywamy elementami.

Można to porównać do:

- pocięcia skomplikowanego kształtu na małe kawałki,
- pokrycia powierzchni drobną siatką,
- zbudowania większego obrazu z wielu małych płytek.

Każdy mały kawałek łatwiej opisać matematycznie.

Potem wyniki z tych małych kawałków składa się w większą całość.

5. Co składa się na jeden element FEM

5.1. Element

Element to mały fragment obszaru obliczeniowego.

W tej pracy interesują nas między innymi elementy pryzmatyczne.

Nie trzeba znać ich dokładnej geometrii, żeby zrozumieć ideę: to po prostu jeden z typów „cegielek”, z których złożona jest siatka.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

5.2. Węzły

Każdy element ma charakterystyczne punkty, zwane węzłami.

Można myśleć o nich jak o punktach zaczepienia, w których opisujemy stan rozwiązania.

5.3. Funkcje kształtu

Żeby opisać, co dzieje się wewnątrz elementu, używa się funkcji kształtu.

To matematyczne funkcje, które pozwalają wyznaczyć zachowanie wnętrza elementu na podstawie wartości w jego węzłach.

W uproszczeniu:

- nie liczymy wszystkiego w nieskończenie wielu punktach,
- tylko budujemy inteligentne przybliżenie wewnątrz elementu.

5.4. Punkty całkowania

W wielu obliczeniach FEM trzeba policzyć wkład z wnętrza elementu.

Zamiast robić to dokładnie w każdym punkcie, wybiera się specjalne punkty pomocnicze, w których obliczenie jest wykonywane.

To są punkty całkowania albo punkty Gaussa.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Można je traktować jak mądrze dobrane punkty kontrolne.

6. Czym są macierz i wektor w tym projekcie

Dla osoby spoza matematyki te słowa potrafią brzmieć odstraszaająco, ale intuicja jest dość prosta.

6.1. Wektor

Wektor to uporządkowana lista liczb.

Można go traktować jak kolumnę wartości.

W FEM wektor często opisuje:

- wymuszenia,
- źródła,
- wpływy zewnętrzne,
- albo część stanu układu.

6.2. Macierz

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Macierz to prostokątna tabela liczb.

Każda liczba mówi, jak jeden fragment układu wpływa na inny.

Najprościej:

- wektor to lista,
- macierz to tabela zależności.

6.3. Lokalna macierz i lokalny wektor

Dla pojedynczego elementu komputer wylicza zwykle:

- lokalną macierz,
- lokalny wektor prawej strony.

Intuicyjnie:

- lokalna macierz opisuje zależności wewnątrz małego fragmentu,
- lokalny wektor opisuje, jakie działają tam wymuszenia lub źródła.

Dopiero potem te lokalne obiekty składa się w większy układ globalny.

7. Dlaczego jeden mały element jest ważny dla wydajności

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Bo takich elementów w realnym problemie może być bardzo dużo.

To oznacza, że jeśli pojedynczy element liczy się trochę szybciej, to po przemnożeniu przez tysiące lub miliony elementów zysk staje się bardzo duży.

Dlatego nawet pozornie niewielkie różnice w organizacji kernela FEM mają znaczenie praktyczne.

8. Co to jest kernel

W tym projekcie bardzo często pojawia się słowo kernel.

Najprościej można o nim myśleć jak o małym, wyspecjalizowanym fragmencie programu, który wykonuje powtarzalną pracę obliczeniową.

Na GPU kernel jest uruchamiany równolegle w wielu kopiach naraz.

Czyli:

- kernel to nie cały program,
- kernel to najważniejszy fragment roboczy, w którym dzieje się właściwe liczenie.

W kontekście v3 kernel FEM to fragment odpowiedzialny za policzenie wkładów elementowych.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

9. CPU i GPU - intuicja bez żargonu

9.1. CPU

CPU jest bardziej uniwersalne.

Dobrze radzi sobie z:

- złożoną logiką,
- wieloma różnymi zadaniami,
- nieregularnym sterowaniem,
- sytuacjami, w których trzeba często podejmować decyzje.

Można o nim myśleć jak o małej grupie bardzo wszechstronnych specjalistów.

9.2. GPU

GPU zostało zaprojektowane do masowego wykonywania wielu podobnych operacji równocześnie.

Można o nim myśleć jak o bardzo dużej grupie pracowników, z których każdy wykonuje prostszy fragment tej samej pracy.

GPU jest bardzo silne wtedy, gdy:

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- pracę można dobrze podzielić,
- wiele operacji wygląda podobnie,
- dane są ułożone w sposób przyjazny dla pamięci,
- narzut organizacyjny nie dominuje nad samym liczeniem.

10. Co to jest pamięć i dlaczego jest tak ważna

Wydajność obliczeń nie zależy tylko od tego, jak szybko układ liczy.

Bardzo często zależy od tego, jak szybko potrafi pobrać dane i zapisać wynik.

Można to porównać do kuchni:

- nawet najlepszy kucharz nic nie zrobi szybko, jeśli co chwilę musi biec do odległego magazynu po składniki,
- jeśli składniki są dobrze przygotowane i ułożone, praca przyspiesza.

W świecie obliczeń komputerowych bardzo często to właśnie ruch danych, a nie samo liczenie, staje się prawdziwym ograniczeniem.

11. Co to jest backend

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Backend w tym projekcie oznacza praktyczny sposób wykonania obliczenia na danej platformie.

To warstwa, która odpowiada na pytania:

- jak przekazać dane do urządzenia,
- jak uruchomić kernel,
- jak zbudować kod dla danej platformy,
- jak odebrać wynik.

Przykładowe backendy w v3 to:

- cpu,
- opencl,
- metal.

To ważne rozróżnienie.

Ten sam problem matematyczny może być uruchomiony przez różne backendy.

Wtedy logika zadania pozostaje podobna, ale sposób wykonania i koszty sprzętowe mogą być zupełnie inne.

12. Co to znaczy, że obliczenie jest równoległe

Obliczenie równoległe to takie, w którym wiele fragmentów pracy wykonuje się jednocześnie.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Jeśli zadanie da się podzielić na dużą liczbę podobnych fragmentów, można je rozdać wielu jednostkom wykonawczym naraz.

To właśnie jest jedna z głównych przewag GPU.

Ale sama równoległość nie gwarantuje sukcesu.

Trzeba jeszcze zadbać o:

- podział pracy,
- organizację pamięci,
- odpowiednią wielkość grup roboczych,
- ograniczenie zbędnych synchronizacji.

13. Co to jest workgroup

Na GPU praca jest organizowana w grupy robocze.

Można je traktować jak małe zespoły pracowników wykonujących pokrewny fragment zadania.

Sposób zorganizowania tych zespołów wpływa na:

- współdzielenie pamięci,
- koszt komunikacji,
- możliwość ponownego użycia danych,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- narzut sterowania.

Dlatego parametry grup roboczych często istotnie wpływają na wynik benchmarku.

14. Czym jest benchmark

Benchmark to uporządkowany test, który ma coś zmierzyć.

W tym projekcie benchmark nie jest po prostu pojedynczym programem.

To raczej cała procedura pomiarowa, która odpowiada na pytanie:

- jak dany system zachowuje się w określonych warunkach?

Benchmark może mierzyć:

- czas,
- przepustowość,
- liczbę operacji na sekundę,
- zużycie energii,
- stabilność,
- zgodność wyniku.

15. Czym jest mikrobenchmark

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Mikrobenchmark to bardzo mały, celowo uproszczony benchmark.

Jego zadaniem nie jest udawanie całej aplikacji, tylko zbadanie jednego konkretnego mechanizmu.

Można to porównać do medycyny.

Zamiast od razu opisywać cały organizm, wykonuje się konkretne badanie, które sprawdza jeden wybrany aspekt.

Mikrobenchmarki w v3 badają między innymi:

- przepustowość pamięci,
- opóźnienie dostępu do pamięci,
- przepustowość obliczeniową,
- wrażliwość na wzorzec odczytu i zapisu,
- skutki różnych organizacji pracy.

15.1. Dlaczego mikrobenchmarki są potrzebne

Bo same czasy końcowej aplikacji są zbyt mało wyjaśniające.

Jeśli widzimy, że jeden backend jest szybszy, a drugi wolniejszy, to bez dodatkowej wiedzy nie wiemy:

- czy ogranicza go pamięć,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- czy ogranicza go liczenie,
- czy problemem jest organizacja danych,
- czy narzut uruchomienia,
- czy niewłaściwy podział pracy.

Mikrobenchmarki tworzą więc warstwę diagnostyczną.

15.2. Dlaczego same mikrobenchmarki nie wystarczają

Bo są zbyt małe i zbyt czyste.

Nie reprezentują w pełni realistycznej aplikacji.

Dlatego v3 nie kończy się na mikrobenchmarkach. Traktuje je jako pierwszą warstwę wyjaśniającą, a następnie sprawdza, czy te same obserwacje mają sens również dla realistycznego kernela FEM.

16. Najważniejsze pojęcia wydajnościowe

16.1. Przepustowość pamięci

To odpowiedź na pytanie:

- jak dużo danych można przenieść w jednostce czasu?

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Jeśli obliczenie ciągle czeka na dane z pamięci, to nawet bardzo silny układ obliczeniowy nie pokaże pełnej mocy.

16.2. Opóźnienie

To odpowiedź na pytanie:

- jak długo trzeba czekać na pojedynczą reakcję lub pojedynczy dostęp?

Można mieć wysoką przepustowość, ale wciąż odczuwać koszt pojedynczych operacji.

16.3. Throughput obliczeniowy

To odpowiedź na pytanie:

- ile pracy obliczeniowej można wykonać w jednostce czasu?

Jeśli problem jest „bogaty obliczeniowo”, ten parametr staje się bardzo ważny.

16.4. Wąskie gardło

Wąskie gardło to element systemu, który realnie ogranicza szybkość całości.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Może nim być:

- pamięć,
- przepustowość zapisu,
- zbyt mały równoległy podział pracy,
- narzut uruchomienia,
- albo samo liczenie.

Jednym z głównych celów v3 jest właśnie ustalenie, gdzie to wąskie gardło się znajduje.

16.5. Roofline

Roofline to model, który pomaga odróżnić dwa główne rodzaje ograniczeń:

- ograniczenie przez pamięć,
- ograniczenie przez moc obliczeniową.

Nie trzeba znać całej matematyki tego modelu, żeby uchwycić sens.

Roofline to po prostu sposób patrzenia na problem przez pytanie:

- czy bardziej brakuje nam danych,
- czy bardziej brakuje nam mocy liczenia?

17. Dlaczego kernel może mieć różne opcje organizacyjne

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Realistyczny kernel FEM można wykonać na różne sposoby, nie zmieniając jego celu matematycznego.

To trochę jak gotowanie według tego samego przepisu, ale z inną organizacją kuchni:

- można składniki pobierać z magazynu za każdym razem,
- można część z nich przygotować wcześniej,
- można trzymać je bliżej siebie,
- można coś przechowywać chwilowo, żeby użyć ponownie,
- można coś policzyć jeszcze raz zamiast to przechowywać.

W v3 i w warstwie referencyjnej pojawiają się więc różne opcje organizacyjne.

Najważniejsze z nich to:

17.1. coal_read

To dotyczy sposobu odczytu danych.

Intuicyjnie:

- czy wiele wątków pobiera dane w uporządkowany sposób,
- czy raczej każdy sięga gdzie indziej.

Uporządkowany odczyt zwykle bardziej odpowiada sprzętowi.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

17.2. coal_write

To samo, ale dla zapisu wyników.

Zorganizowany zapis jest zwykle łatwiejszy dla pamięci niż zapis chaotyczny.

17.3. compute_all_shape_fun_der

To decyzja typu:

- czy pewne wartości lepiej policzyć na miejscu,
- czy lepiej odczytać je z pamięci.

To klasyczny kompromis:

- więcej liczenia,
- albo więcej ruchu pamięciowego.

Różne architektury mogą preferować różne strony tego kompromisu.

17.4. workspace_*

To dotyczy użycia dodatkowej pamięci roboczej.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Można ją porównać do wspólnego notesu, z którego korzysta mały zespół.

Zalety:

- można coś przechować lokalnie i użyć ponownie.

Wada:

- taki notes ma ograniczoną pojemność,
- a korzystanie z niego też kosztuje.

17.5. padding

Padding to celowe dodawanie odstępów lub pustych miejsc w danych.

Brzmi dziwnie, ale czasem pomaga sprzętowi lepiej organizować pamięć.

To trochę jak zostawienie luzu na półce, żeby szybciej wyjmować przedmioty.

18. Czym jest fem_option_validation

To bardzo ważna warstwa v3.

Jej rola jest pośrednia między:

- prostymi mikrobenchmarkami,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- a pełnym, realistycznym kernelem FEM.

Dlaczego jest potrzebna?

Bo mikrobenchmarki są zbyt proste, a pełny kernel jest na tyle złożony, że trudniej jednoznacznie powiedzieć, co naprawdę zadecydowało o wyniku.

fem_option_validation działa jak most.

Sprawdza:

- jak backend reaguje na wybrane, kontrolowane zmiany,
- które są bardzo bliskie temu, co dzieje się później w realistycznym kernelu.

To znaczy:

- nie jest to jeszcze pełna aplikacja,
- ale nie jest to też już tylko prymitywny test jednej operacji.

W praktyce ta warstwa pomaga odpowiedzieć na pytanie:

- czy obserwacja z mikrobenchmarku pozostaje sensowna, kiedy obliczenie zaczyna przypominać prawdziwy kernel FEM?

19. Czym jest kampania referencyjna exact

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Warstwa `reference_exact` to referencyjny przebieg oparty o oryginalną implementację OpenCL.

Ważne: słowo `exact` nie oznacza tutaj matematycznej doskonałości w sensie absolutnym.

Oznacza raczej:

- możliwie wierne odtworzenie referencyjnej, historycznej ścieżki wykonania.

Ta warstwa daje nam:

- referencyjny sposób liczenia,
- referencyjne dane wejściowe dla kernela,
- referencyjny wynik,
- referencyjne metadane uruchomienia.

To jest coś w rodzaju wzorca laboratoryjnego.

20. Dlaczego nie wystarczy tylko ten sam plik problemu

Na pierwszy rzut oka mogłoby się wydawać, że jeśli dwa backendy dostaną ten sam plik siatki i ten sam plik problemu, to dostają te same dane.

W praktyce to jeszcze za mało.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Po drodze system buduje bardziej szczegółowe bufory runtime, czyli dane przygotowane już konkretnie dla kernela.

To właśnie one decydują o tym, co kernel naprawdę dostaje do policzenia.

Dlatego v3 zamraża nie tylko plik wysokiego poziomu, ale rzeczywiste dane wejściowe dla kernela.

To jest warunek uczciwego porównania.

21. Czym jest replay na zamrożonych danych wejściowych

To jedna z najważniejszych idei całego systemu.

Frozen-input replay oznacza:

- bierzemy dokładnie te same dane wejściowe do obliczenia,
- zapisujemy je,
- a potem uruchamiamy je na innym backendzie.

Czyli:

- nie generujemy nowych danych,
- nie liczymy „podobnego” przypadku,
- tylko odtwarzamy to samo wejście.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Dzięki temu możemy sprawdzić:

- czy inny backend liczy to samo,
- a nie tylko czy daje podobny czas.

21.1. Po co to jest potrzebne

Bo bez tego zawsze można postawić zarzut:

- może jeden backend był szybszy, ale tak naprawdę liczył coś trochę innego.

Replay zamyka ten problem.

22. Czym jest expected output

Expected output to oczekiwany, referencyjny wynik obliczenia.

Jeśli mamy:

- zamrożone dane wejściowe,
- oraz zapisany wynik referencyjny,

możemy sprawdzić, czy backend docelowy daje ten sam wynik w granicach dopuszczalnej różnicy numerycznej.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

To właśnie jest warstwa walidacji poprawności.

23. Dlaczego wyniki nie zawsze muszą być bit po bicie identyczne

Komputery liczą na liczbach zmiennoprzecinkowych.

To oznacza, że bardzo drobne różnice mogą pojawiać się naturalnie, nawet jeśli obliczenie jest logicznie to samo.

Przyczyny mogą być różne:

- inna kolejność operacji,
- inne optymalizacje kompilatora,
- inne wykorzystanie instrukcji sprzętowych,
- różne szczegóły wykonania na dwóch platformach.

Dlatego poprawności nie ocenia się wyłącznie przez pytanie:

- czy każdy bit jest identyczny?

Częściej ocenia się przez:

- maksymalną różnicę bezwzględną,
- błąd średniokwadratowy,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- liczbę przypadków mieszczących się w tolerancji.

To jest podejście uczciwe naukowo.

24. Czym jest profiler

Profiler to narzędzie, które zagląda do środka wykonania programu.

Jeśli benchmark odpowiada na pytanie:

- „jak szybko?”

To profiler pomaga odpowiedzieć:

- „dlaczego akurat tak szybko albo tak wolno?”.

Profiler może pokazywać między innymi:

- ile czasu zajął dany kernel,
- czy obliczenie czekało na pamięć,
- czy sprzęt był dobrze wykorzystany,
- czy problemem był odczyt, zapis albo organizacja pracy.

To czyni profilery bardzo ważnym łącznikiem między teorią a praktyką.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

25. Czym jest profiler_correlation

To warstwa, która spina trzy rodzaje wiedzy:

- wiedzę z mikrobenchmarków,
- wiedzę z fem_option_validation,
- wiedzę z realistycznego kernela i raportów profilerowych.

Można to rozumieć tak:

1. mikrobenchmarki mówią, co architektura lubi albo czego nie lubi,
2. walidacja FEM sprawdza to na bardziej realistycznych próbach,
3. profiler sprawdza, czy w prawdziwym kernelu widać zgodny mechanizm,
4. raport korelacyjny próbuje połączyć te trzy warstwy w jedną interpretację.

To jest bardzo wartościowe, bo pozwala przejść od:

- „tak wyszło w pomiarze”

do:

- „wiemy, co najprawdopodobniej spowodowało ten wynik”.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

26. Czym są provenance i hashe

W badaniach eksperymentalnych bardzo ważne jest pytanie:

- czy da się odtworzyć ten sam wynik?

Do tego potrzebne są informacje o pochodzeniu artefaktów, czyli provenance.

W v3 oznacza to między innymi:

- jaki plik powstał,
- kiedy powstał,
- w jakim środowisku,
- z jakiego uruchomienia,
- i czy na pewno nie został po drodze zmieniony.

26.1. Hash

Hash można traktować jak cyfrowy odcisk palca pliku.

Jeśli plik się zmieni, jego hash też się zmieni.

Dzięki temu można sprawdzić:

- czy dwa pliki są dokładnie tym samym artefaktem,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- czy wynik jest rzeczywiście tym, na który się powołujemy.

W kontekście doktoratu bardzo wzmacnia to wiarygodność pracy.

27. Dlaczego powtórzenia i statystyka są konieczne

Komputer nie pracuje w idealnej próżni.

Na wynik czasu mogą wpływać:

- inne procesy w tle,
- nagrzanie sprzętu,
- chwilowe zmiany obciążenia,
- zachowanie systemu operacyjnego,
- drobna niestabilność uruchomień.

Dlatego pojedynczy przebieg prawie nigdy nie powinien być jedyną podstawą wniosku.

Potrzebne są:

- powtórzenia,
- średnie,
- rozrzut wyników,
- czasem przedziały niepewności.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

To pozwala odróżnić:

- prawdziwy efekt,
- od jednorazowego przypadku.

28. Jak czytać wykresy w takim projekcie

Dla osoby spoza informatyki wykresy wydajności często wyglądają groźnie, ale zwykle odpowiadają na dość proste pytania.

28.1. Wykres słupkowy

Najczęściej pokazuje porównanie kilku wariantów.

Pytanie brzmi wtedy zwykle:

- który wariant był szybszy,
- który zużył mniej energii,
- który miał mniejszy błąd.

28.2. Wykres trendu

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Pokazuje, jak wynik zmienia się wraz ze wzrostem parametru.

Na przykład:

- liczby elementów,
- rozmiaru grupy roboczej,
- liczby punktów całkowania.

28.3. Wykres korelacyjny

Pokazuje, czy dwie rzeczy „idą razem”.

Na przykład:

- czy backend, który ma dobrą przepustowość pamięci, rzeczywiście lepiej radzi sobie z wariantami opartymi o uporządkowany odczyt.

W badaniach ważne jest jednak, żeby nie mylić korelacji z dowodem przyczyny.

Korelacja wspiera interpretację, ale nie zastępuje całej metodologii.

29. Jak warstwy v3 układają się w jedną historię

Najprostsza mapa logiczna wygląda tak:

1. Mikrobenchmarki

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- mówią, jakie są podstawowe cechy sprzętu.

2. Walidacja opcji FEM

- sprawdza, czy te cechy mają znaczenie dla wzorców bliskich rzeczywistemu kernelowi FEM.

3. Reference exact

- dostarcza referencyjnego sposobu liczenia i zamrożonych danych wejściowych.

4. Replay poprawności

- sprawdza, czy inny backend liczy to samo na tych samych danych.

5. Korelacja profilerowa

- scala wiedzę z poprzednich warstw i pomaga wyjaśniać wyniki.

To oznacza, że v3 nie jest tylko zbiorem testów.

To jest uporządkowana metodologia badawcza.

30. Co można twierdzić naukowo, a czego nie

30.1. Co można twierdzić mocno

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Jeśli:

- mikrobenchmarki wskazują określone ograniczenie,
- fem_option_validation pokazuje zgodny trend,
- profiler pokazuje zgodny mechanizm,
- replay potwierdza zgodność obliczeń,

to można bardzo mocno twierdzić, że:

- interpretacja architektoniczna ma sens,
- a porównanie backendów dotyczy rzeczywiście tego samego obliczenia.

30.2. Czego nie należy twierdzić zbyt mocno

Nie powinno się twierdzić, że:

- sam podobny czas oznacza poprawne obliczenie,
- sama zgodność wykresu wydajności oznacza zgodność matematyczną,
- sama natywna kampania wydajnościowa jest dowodem ścisłej zgodności 1:1 wobec referencji.

To byłyby zbyt mocne uproszczenia.

31. Dlaczego to ma sens pod doktorat

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Dla doktoratu ważne jest nie tylko to, że system działa.

Ważniejsze jest to, że:

- ma jasną logikę,
- rozdziela role poszczególnych warstw,
- daje możliwość kontroli poprawności,
- umożliwia interpretację wyników,
- i pozostawia artefakty, które można odtworzyć i zarchiwizować.

To czyni z v3 nie tylko narzędzie do uruchamiania testów, ale realną platformę badawczą.

32. Słownik najważniejszych pojęć

Architektura

Sposób zbudowania i organizacji sprzętu komputerowego.

Backend

Warstwa wykonawcza, która uruchamia obliczenie na konkretnej platformie.

Benchmark

Uporządkowany test mierzący określone zachowanie systemu.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Replay poprawności

Uruchomienie tego samego obliczenia na tych samych zamrożonych danych wejściowych w celu sprawdzenia poprawności wyniku.

Expected output

Referencyjny wynik, z którym porównuje się wynik backendu docelowego.

FEM

Metoda elementów skończonych - sposób przybliżania rozwiązań problemów inżynierskich przez podział obszaru na małe elementy.

Frozen inputs

Zamrożone dane wejściowe do kernela, zapisane tak, aby dało się odtworzyć dokładnie to samo obliczenie.

GPU

Procesor wyspecjalizowany w masowym wykonywaniu wielu podobnych operacji równolegle.

Hash

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Cyfrowy odcisk palca pliku.

Kernel

Najważniejszy fragment obliczeniowy wykonywany wielokrotnie, często równolegle.

Lokalna macierz

Macierz opisująca zależności wewnątrz pojedynczego elementu FEM.

Mikrobenchmark

Mały benchmark badający jeden wybrany mechanizm sprzętowy lub wykonawczy.

Profiler

Narzędzie pokazujące, gdzie program spędza czas i co go ogranicza.

Provenance

Informacja o pochodzeniu artefaktu: skąd pochodzi, kiedy powstał i w jakim środowisku.

Replay bundle

Pakiet danych potrzebnych do odtworzenia obliczenia na innym backendzie.

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

Tolerancja

Dopuszczalny poziom drobnej różnicy numerycznej między dwoma wynikami.

Węzeł

Charakterystyczny punkt elementu FEM, w którym opisuje się wartości rozwiązania.

Workgroup

Grupa robocza w modelu równoległego wykonania na GPU.

33. Najkrótsze podsumowanie dla osoby spoza informatyki

Jeśli trzeba to powiedzieć w jednym zdaniu, najuczciwiej brzmi to tak:

- v3 służy do sprawdzania, dlaczego to samo obliczenie inżynierskie zachowuje się różnie na różnych komputerach, przy jednoczesnym pilnowaniu, żeby porównywane systemy rzeczywiście liczyły to samo.

A jeśli w dwóch zdaniach:

- mikrobenchmarki mówią, jakie są możliwości i ograniczenia sprzętu,

Teoria od podstaw

Szerokie wyjaśnienie pojęć i warstw projektu dla osoby spoza informatyki.

- a warstwy FEM, replayu i korelacji profilerowej sprawdzają, czy te same obserwacje pozostają prawdziwe dla realistycznego obliczenia.

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

Plik źródłowy: docs/ROZDZIAL_01_MOTYWACJA_I_KONTEKST.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

1.1. Problem ogólny

Współczesne obliczenia naukowe i inżynierskie są coraz silniej uzależnione od wydajności sprzętu komputerowego. Ten sam model matematyczny może być uruchamiany na:

- klasycznym procesorze CPU,
- procesorze graficznym GPU,
- różnych warstwach wykonawczych i backendach,
- różnych sterownikach, runtime'ach i systemach operacyjnych.

W praktyce prowadzi to do sytuacji, w której:

- to samo zadanie obliczeniowe daje różne czasy wykonania,
- różne platformy premiuje różne strategie organizacji kernela,
- a interpretacja tych różnic nie jest oczywista, jeśli patrzy się wyłącznie na czas końcowy.

To właśnie jest podstawowy problem badawczy rozwiązywany przez v3.

1.2. Dlaczego sama analiza czasu jest niewystarczająca

W wielu pracach wydajnościowych stosuje się prosty schemat:

- uruchomić program,

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

- zmierzyc czas,
- porównac wyniki między platformami.

Taki schemat jest użyteczny, ale ma ograniczenia.

Sam czas nie odpowiada na pytania:

- dlaczego dana architektura jest szybsza lub wolniejsza,
- czy w obu przypadkach wykonywane jest rzeczywiście to samo obliczenie,
- czy przewaga wydajności wynika z mocy obliczeniowej, organizacji pamięci czy narzutu wykonania,
- czy obserwowany wynik ma charakter stabilny, czy jest tylko przypadkowym artefaktem środowiska.

W konsekwencji analiza oparta wyłącznie na czasach bywa zbyt słaba interpretacyjnie.

1.3. Potrzeba warstwy wyjaśniającej

Aby porównanie wydajności miało wartość naukową, potrzebna jest warstwa, która pozwala zrozumieć mechanizm stojący za wynikiem.

W v3 te role pełni:

- mikrobenchmarki architektoniczne,
- warstwa `fem_option_validation`,

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

- korelacja profilerowa,
- replay poprawności.

Ich wspólnym celem nie jest tylko wygenerowanie kolejnych wykresów, ale zbudowanie logicznego ciągu dowodowego:

1. architektura ma określone ograniczenia,
2. ograniczenia te można zmierzyć,
3. ograniczenia te powinny wpływać na realistyczny kernel w określony sposób,
4. wynik obliczenia pozostaje poprawny mimo zmiany backendu,
5. zatem interpretacja wydajności nie jest przypadkowa.

1.4. Rola benchmarku realistycznego

W pracy wykorzystuje się realistyczny kernel FEM jako warstwę walidacji aplikacyjnej. Jego rola nie polega na byciu głównym wkładem pracy, lecz na sprawdzeniu, czy obserwacje wynikające z mikrobenchmarków rzeczywiście mają znaczenie dla obliczenia zbliżonego do zastosowań praktycznych.

To rozroznienie jest bardzo ważne:

- mikrobenchmarki odpowiadają za diagnozę architektury,
- realistyczny kernel odpowiada za sprawdzenie, czy diagnoza ma sens poza prostym testem

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

syntetycznym.

1.5. Główna idea metodologiczna v3

v3 opiera się na następującym założeniu:

- wiarygodne badanie wydajności nie powinno oddzielać czasu od poprawności i od interpretacji.

Dlatego platforma rozdziela pracę na kilka warstw:

1. mikrobenchmarki - opis podstawowych możliwości sprzętu,
2. Walidacja opcji FEM - warstwa pomostowa bliska realistycznemu kernelowi,
3. referencyjny exact - referencyjny tor wykonania,
4. replay poprawności - walidacja, czy inne backendy liczą to samo,
5. korelacja profilerowa - połączenie danych pomiarowych z rzeczywistym zachowaniem kernela.

1.6. Wkład metodologiczny

Wkład pracy nie polega na prostym uruchomieniu istniejącego benchmarku. Wkład polega na zbudowaniu metodologii, w której:

- mikrobenchmarki nie są odizolowane od aplikacji,
- benchmark realistyczny nie jest traktowany jako jedyny dowód,

Rozdział 1. Motywacja i kontekst badań

Dlaczego sama analiza czasu nie wystarcza i jaki problem badawczy rozwiązuje v3.

- replay poprawności wymusza porównanie tego samego obliczenia na tych samych danych,
- a profiler służy jako narzędzie łączące obserwacje niskopoziomowe z zachowaniem realnego kernela.

1.7. Znaczenie dla rozprawy

W takim układzie doktorat nie jest tylko raportem z wyników wydajnościowych. Staje się pracą o metodzie badania zależności między:

- architektura sprzętu,
- organizacja obliczenia,
- poprawnością wyniku,
- i interpretacja zachowania realistycznego kernela numerycznego.

To nadaje pracy silniejszy charakter naukowy i lepiej uzasadnia jej wartość poznawczą.

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela

Plik źródłowy: docs/ROZDZIAL_02_OBLICZENIA_NUMERYCZNE_I_FEM.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

2.1. Obliczenia numeryczne jako przybliżanie rozwiązań

W naukach stosowanych wiele problemów opisuje się równaniami różniczkowymi. Rzeczywiste geometrie, warunki brzegowe oraz współczynniki materiałowe sprawiają jednak, że dla dużej klasy takich problemów nie istnieje wygodna postać analityczna rozwiązania. Wtedy stosuje się metody numeryczne.

Ich istota polega na tym, że komputer nie znajduje idealnego wzoru na rozwiązanie, lecz buduje jego przybliżenie przez dyskretyzację problemu i wykonanie dużej liczby obliczeń elementarnych.

2.2. Intuicja metody elementów skończonych

Metoda elementów skończonych (FEM) należy do najważniejszych technik dyskretyzacji problemów ciągłych. Jej idea polega na podziale badanego obszaru na skończoną liczbę małych elementów. Zamiast rozwiązywać problem globalnie i jednorazowo, rozważa się wiele małych fragmentów, dla których można sformułować lokalne zależności matematyczne.

W ten sposób:

- rzeczywisty obszar zastępuje się siatką elementów,
- pole rozwiązania aproksymuje się funkcjami kształtu,

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

- a lokalne układy składa się potem w układ globalny.

2.3. Element, węzeł i funkcja kształtu

Podstawowymi obiektami FEM są:

- element - mały fragment geometrii,
- węzeł - charakterystyczny punkt elementu,
- funkcja kształtu - funkcja łącząca wartości w węzłach z wartością wewnątrz elementu.

W praktyce to oznacza, że rozwiązanie nie jest przechowywane wszędzie, lecz w wybranych punktach, a wartości pomiędzy nimi są rekonstruowane przez aproksymację.

2.4. Całkowanie numeryczne i punkty Gaussa

W wielu formułacjach FEM trzeba policzyć całki po objętości lub powierzchni elementu. W praktyce wykonuje się to przez całkowanie numeryczne, zwykle z użyciem punktów Gaussa. Dla każdego punktu obliczane są odpowiednie składniki funkcji kształtu, pochodnych, geometrii i współczynników równania.

Z punktu widzenia wydajności jest to ważne, bo:

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

- liczba punktów całkowania bezpośrednio wpływa na liczbę operacji,
- dane potrzebne w tych punktach generują charakterystyczne wzorce odczytu i zapisu,
- a organizacja ich obsługi jest jednym z głównych tematów strojenia kernela.

2.5. Lokalna macierz i lokalny wektor

Dla pojedynczego elementu FEM buduje się zwykle lokalna macierz i lokalny wektor prawej strony. Lokalna macierz opisuje relacje pomiędzy stopniami swobody elementu, natomiast lokalny wektor odpowiada za składniki wymuszające lub źródłowe.

To właśnie na poziomie budowy tych lokalnych obiektów wykonuje się dużą część kosztownych obliczeń w realistycznym kernelu.

2.6. Dlaczego kernel elementowy jest dobrym obiektem badań wydajności

Kernel elementowy ma kilka cech, które czynią go bardzo dobrym obiektem badań:

- jest obliczeniowo intensywny,
- intensywnie korzysta z pamięci,
- daje się wykonać bardzo wiele razy dla wielu elementów,
- a jego organizacja może być zmieniana bez zmiany celu matematycznego obliczenia.

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

To oznacza, że nawet drobne różnice w organizacji pracy przekładają się na mierzalne różnice czasowe i energetyczne.

2.7. Co tak naprawdę porównujemy w warstwie FEM

W analizie v3 nie chodzi o spor między dwiema różnymi fizykami. Chodzi o porównanie różnych sposobów wykonania tego samego typu obliczenia elementowego.

Zmieniać mogą być m.in.:

- wzorce odczytu i zapisu,
- decyzja o ponownym liczeniu lub przechowywaniu danych,
- wykorzystanie pamięci roboczej,
- rozkład pracy między wątki,
- uporządkowanie danych w pamięci.

Każda z tych decyzji nie musi zmieniać tego, co liczymy, ale może zmieniać to, jak drogo sprzętowo wykonywane jest obliczenie.

2.8. Znaczenie tego rozdziału dla metodologii

Rozdział 2. Obliczenia numeryczne i FEM

Podstawy matematyczne i obliczeniowe potrzebne do zrozumienia realistycznego kernela FEM.

Ten rozdział uzasadnia, dlaczego realistyczny kernel FEM nie jest jedynie dodatkowym benchmarkiem. Jest on reprezentantem rzeczywistego obciążenia obliczeniowego, na którym można sprawdzać, czy obserwacje z poziomu architektury i mikrobenchmarków pozostają prawdziwe w problemie o znaczeniu praktycznym.

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

Plik źródłowy: docs/ROZDZIAL_03_ARCHITEKTURA_I_BACKENDY.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisanie rozprawy.

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

3.1. Dlaczego ta sama aplikacja nie zachowuje się tak samo wszędzie

Program nie wykonuje się w próżni. Jego zachowanie zależy od:

- architektury CPU lub GPU,
- organizacji pamięci,
- szerokości i liczby jednostek wykonawczych,
- narzutu uruchomienia,
- warstwy backendowej,
- sterownika i runtime'u.

To oznacza, że to samo zadanie może mieć inne wąskie gardło na różnych platformach.

3.2. CPU a GPU

CPU dobrze radzi sobie z:

- złożona logika,
- nieregularnym przepływem sterowania,
- mniejsza liczba bardziej zróznicowanych zadań.

GPU dobrze radzi sobie z:

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

- ogromna liczba podobnych operacji,
- praca silnie równoległa,
- dobrze zorganizowanym dostępem do pamięci,
- powtarzalna struktura kerneli.

Dla kernela FEM oznacza to, że GPU może być bardzo efektywne, ale tylko wtedy, gdy organizacja danych i pracy jest zgodna z charakterem architektury.

3.3. Pamięć jako główny współuczestnik wydajności

W wielu zastosowaniach wydajność nie jest ograniczona przez samo liczenie, ale przez sposób dostępu do danych. Dwa kluczowe pojęcia to:

- przepustowość - ile danych można przenieść w jednostce czasu,
- opóźnienie - jak długo trwa pojedynczy dostęp.

W praktyce oznacza to, że program może być wolny nie dlatego, że liczy za mało, lecz dlatego, że zbyt długo czeka na dane.

3.4. Backend jako warstwa wykonawcza

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

Backend to warstwa, która mapuje obliczenie na konkretne środowisko wykonawcze. Ten sam problem matematyczny może być uruchamiany przez:

- backend CPU,
- backend OpenCL,
- backend Metal,
- inne backendy GPU.

Backend decyduje o tym:

- jak przygotować dane,
- jak skompilować kernel,
- jak uruchomić obliczenie,
- i jak odebrać rezultat.

3.5. Wzorce dostępu do pamięci i ich znaczenie

Różnice wydajności bardzo często wynikają z tego, jak wiele wątków sięga po dane. W szczególności:

- uporządkowany, współbieżny odczyt jest zwykle bardziej przyjazny,
- chaotyczny lub rozproszony zapis bywa droższy,
- ponowne wykorzystanie danych może być korzystne, ale wymaga miejsca i dobrej organizacji.

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

Własnie z takich powodow opcje kernela typu `coal_read`, `coal_write`, `workspace_*` i `padding` nie sa detalami implementacyjnymi bez znaczenia, lecz rzeczywistymi decyzjami architektonicznymi.

3.6. Obliczenia vs ruch danych

Jedna z klasycznych osi interpretacji brzmi:

- czy program jest ograniczony przez moc obliczeniowa,
- czy przez ruch danych?

Jesli ograniczeniem jest pamiec, zwiekszanie mocy obliczeniowej nie pomoze. Jesli ograniczeniem jest moc obliczeniowa, sama poprawa organizacji pamieci moze nie wystarczyc.

To rozroznienie jest fundamentem modeli typu roofline oraz analizy profilerowej.

3.7. Dlaczego backend-dependent tuning ma sens

Skoro architektury roznie reaguja na:

- wzorce odczytu,
- wzorce zapisu,
- uklad danych,

Rozdział 3. Architektura i backendy

CPU, GPU, pamięć, backendy i konsekwencje organizacji wykonania dla wydajności.

- wielkosc grup roboczych,
- ilosc pamieci roboczej,

to naturalne jest oczekiwanie, ze najlepsza konfiguracja kernela nie bedzie uniwersalna. Jednym z celow v3 jest zbadanie, czy i jak mozna te zaleznosci przewidywac na podstawie warstwy mikrobenchmarkowej i walidacyjnej.

Rozdział 4. Metodologia badawcza i wal

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

Plik źródłowy: docs/ROZDZIAL_04_METODOLOGIA_BADAWCZA_I_WALIDACJA.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

4.1. Potrzeba metodologii wielowarstwowej

Dobra metodologia wydajnościowa powinna oddzielać od siebie trzy pytania:

1. jakie są możliwości architektury,
2. jak zachowuje się realistyczny kernel,
3. czy porównanie backendów dotyczy tego samego obliczenia.

v3 realizuje to przez kilka warstw eksperymentu.

4.2. Mikrobenchmarki jako warstwa diagnostyczna

Mikrobenchmarki odpowiadają za pomiar podstawowych cech sprzętu, takich jak:

- przepustowość pamięci,
- opóźnienie,
- throughput obliczeniowy,
- wrażliwość na wzorce pracy.

Ich rola nie polega na symulowaniu całej aplikacji, lecz na dostarczaniu czytelnych sygnałów o możliwościach i ograniczeniach platformy.

Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

4.3. Walidacja opcji FEM jako warstwa pomostowa

Warstwa `fem_option_validation` zajmuje miejsce pomiędzy mikrobenchmarkiem a realistycznym kernelem. Jej celem jest sprawdzenie, czy wnioski z prostych testów pozostają prawdziwe dla problemów silnie przypominających właściwe obliczenie FEM.

Dzięki temu można zmniejszyć ryzyko zbyt dalekiego przejścia od prostych pomiarów do bardzo złożonej aplikacji.

4.4. Reference exact jako źródło referencji

Reference exact nie jest tutaj głównym wkładem pracy, lecz punktem odniesienia. Ta warstwa dostarcza:

- referencyjnego przebiegu wykonania,
- zamrożonych danych wejściowych dla kernela,
- referencyjnego wyniku,
- metadanych potrzebnych do odtworzenia obliczenia.

To tworzy laboratorium odniesienia, względem którego można oceniać inne backendy.

Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

4.5. Dlaczego sama zgodność wysokopoziomowego problemu nie wystarczy

Jeśli dwa backendy dostaną ten sam plik siatki i ten sam opis problemu, nie oznacza to jeszcze automatycznie identycznego wejścia do kernela. Po drodze powstają bowiem szczegółowe bufor runtime, przygotowane specjalnie dla danego przebiegu obliczenia.

W związku z tym rzetelne porównanie wymaga zamrożenia nie tylko opisu problemu, lecz także rzeczywistych danych wejściowych dla kernela.

4.6. Replay poprawności

Replay poprawności polega na uruchomieniu innego backendu na dokładnie tych samych danych wejściowych, które zostały wykorzystane w referencji. To pozwala odpowiedzieć na pytanie:

- czy obliczenie zostało zachowane,
- a nie tylko czy czas jest porównywalny.

Replay eliminuje najważniejszy zarzut wobec porównań wieloplatformowych: że różne systemy mogły liczyć coś odrobine innego.

4.7. Tolerancja numeryczna

Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

Porównanie wyników nie musi oznaczać identyczności bit po bicie. W obliczeniach zmiennoprzecinkowych dopuszcza się małe różnice wynikające z porządku operacji i szczegółów wykonania. Dlatego stosuje się metryki takie jak:

- max_abs_diff,
- rms_diff,
- liczba rekordów mieszczących się w tolerancji.

To daje uczciwe kryterium poprawności między backendami.

4.8. Korelacja profilerowa jako warstwa interpretacyjna

Korelacja profilerowa spina dane z trzech źródeł:

- mikrobenchmarków,
- walidacji FEM,
- profilerów i realistycznego kernela.

Jej celem jest przejście od prostego stwierdzenia „wariant A jest szybszy” do interpretacji „wariant A jest szybszy, ponieważ architektura premiuje dany wzorzec organizacji danych i pracy”.

Rozdział 4. Metodologia badawcza i walidacja

Warstwy mikrobenchmarków, walidacji FEM, exact i replayu poprawności.

4.9. Znaczenie metodologiczne

Wspólne zastosowanie tych warstw daje mocniejszy argument naukowy niż pojedynczy benchmark.

Wnioski nie wynikają wtedy z jednej liczby, lecz z uporządkowanego ciągu obserwacji i sprawdzeń:

- pomiar,
- walidacja,
- kontrola poprawności,
- interpretacja mechanizmu.

Rozdział 5. Metryki, profiling i interpret

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

Plik źródłowy: docs/ROZDZIAL_05_METRYKI_PROFILING_I_INTERPRETACJA.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Rozdział 5. Metryki, profiling i interpretacja

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

5.1. Czas jako metryka podstawowa, ale niewystarczająca

Czas wykonania jest najbardziej intuicyjna metryka. Pokazuje, ile trwa wykonanie zadania. Nie odpowiada jednak na pytania:

- co było ograniczeniem,
- czy wynik jest poprawny,
- czy różnice mają charakter stabilny,
- czy szybszy backend osiągnął przewagę dzięki lepszej architekturze czy dzięki zmianie semantyki obliczenia.

Dlatego czas powinien być traktowany jako punkt wyjścia, a nie jedyne kryterium interpretacji.

5.2. Throughput i metryki pochodne

W analizach wydajnościowych stosuje się również metryki typu:

- GFLOP/s,
- GB/s,
- ns na jednostkę pracy,
- energia na jednostkę pracy,
- energy-delay product.

Rozdział 5. Metryki, profiling i interpretacja

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

Każda z nich patrzy na problem z innej strony:

- GFLOP/s pyta o wydajność obliczeniową,
- GB/s pyta o intensywność ruchu danych,
- ns na jednostkę pracy upraszcza porównanie kosztu lokalnego,
- energia pokazuje koszt fizyczny uzyskanego wyniku.

5.3. Stabilność wyników

W badaniach nie wystarczy pokazać jednego najlepszego przebiegu. Konieczne jest uwzględnienie:

- powtórzeń,
- rozrzutu,
- współczynnika zmienności,
- czasem przedziałów ufności.

Dopiero wtedy da się odróżnić realny efekt od szumu pomiarowego.

5.4. Rola profilera

Profilery są narzędziami diagnostycznymi. Pokazują, gdzie program spędza czas i jakie są

Rozdział 5. Metryki, profiling i interpretacja

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

najbardziej prawdopodobne źródła strat wydajności. W kontekście tej pracy profiler może wspierać odpowiedzi na pytania:

- czy kernel jest memory-bound czy compute-bound,
- czy problemem jest odczyt, zapis lub wykorzystanie grup roboczych,
- czy zachowanie realistycznego kernela zgadza się z przewidywaniem wynikającym z mikrobenchmarków.

5.5. Od korelacji do interpretacji

Sama korelacja nie jest jeszcze dowodem przyczynowym. Jest jednak bardzo cenna, gdy jest osadzona w szerszej metodologii.

Jeśli jednocześnie:

- mikrobenchmarki wskazują ograniczenie pamięci,
- walidacja FEM pokazuje wrażliwość na uporządkowany odczyt,
- profiler wskazuje istotne oczekiwanie na pamięć,
- a replay potwierdza zgodność obliczeń,

to interpretacja zyskuje dużo silniejsze podstawy.

Rozdział 5. Metryki, profiling i interpretacja

Jak przejść od liczb do interpretacji mechanizmu działania kernela.

5.6. Ograniczenia interpretacyjne

Należy przy tym zachować ostrożność. Z samych metryk nie wolno wyciągać zbyt daleko idących wniosków. W szczególności:

- podobny czas nie dowodzi poprawności,
- przewaga jednej metryki nie opisuje całego zachowania systemu,
- jeden zestaw danych nie gwarantuje pełnej ogólności,
- a silna korelacja nadal powinna być wspierana logiką metodologiczną.

5.7. Znaczenie dla rozprawy

Rozdział metryczny i profilerowy jest potrzebny po to, aby przejść od wyników liczbowych do sensownej interpretacji. To właśnie w nim spina się:

- warstwa pomiaru,
- warstwa walidacji,
- i warstwa wyjaśnienia mechanizmu.

Dzięki temu rozprawa nie ogranicza się do listy tabel i wykresów, lecz pokazuje, dlaczego wyniki mają taki, a nie inny kształt.

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

Plik źródłowy: docs/ROZDZIAL_06_SYNTEZA_WKLADU_I_ZAKRES_WNIOSKOW.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

6.1. Co jest przedmiotem pracy

Przedmiotem pracy nie jest pojedynczy benchmark ani pojedyncza implementacja kernela.

Przedmiotem pracy jest metodologia badawcza łącząca:

- mikrobenchmarki architektury,
- warstwę walidacji opcji FEM,
- referencyjny przebieg exact,
- replay poprawności na zamrożonych danych wejściowych,
- korelacje profilerowe.

Taka konstrukcja pozwala przechodzić od prostych miar architektury do realistycznego obciążenia obliczeniowego w sposób interpretowalny i kontrolowany.

6.2. Co stanowi wkład własny

Wkład własny pracy można uporządkować w kilku punktach:

1. spójna platforma eksperymentalna łącząca mikrobenchmarki, kampanie FEM i replay poprawności
2. warstwa Walidacja opcji FEM jako pomost między testami syntetycznymi a realistycznym kerneliem,

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

3. replay poprawności pozwalający porównywać backendy na tych samych danych wejściowych i tym samym oczekiwanym wyniku,
4. korelacja wsparta profilerem, która pomaga przejść od pomiaru do interpretacji mechanizmu,
5. uporządkowany protokół eksperymentalny z provenance, hashowaniem i artefaktami gotowymi do archiwizacji.

6.3. Czego praca nie twierdzi

Rzetelność rozprawy wymaga również jasnego wskazania ograniczeń. Praca nie twierdzi, że:

- jeden benchmark realistyczny opisuje całą klasę zastosowań FEM,
- każdy backend ma identyczną semantykę pomiaru czasu,
- korelacja sama w sobie jest dowodem przyczynowym,
- wyniki dla jednej architektury automatycznie generalizują się na wszystkie inne platformy.

Zamiast tego praca pokazuje, w jakim zakresie można budować wiarygodne wnioski i jakie warunki muszą być spełnione, aby porównanie było uczciwe.

6.4. Zakres wniosków

Na podstawie metodologii v3 można formułować wnioski w trzech warstwach:

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

1. architektonicznej - jakie cechy sprzętu ujawniają mikrobenchmarki,
2. algorytmicznej - które grupy opcji FEM są wrażliwe na określone cechy sprzętu,
3. walidacyjnej - czy porównywane backendy liczą to samo na tych samych danych.

Dopiero połączenie tych trzech warstw daje mocniejszy argument naukowy niż sama tabela czasów.

6.5. Dlaczego taka konstrukcja jest silna doktoratowo

Największa siła tej konstrukcji polega na tym, że nie ogranicza się ona do jednego rodzaju dowodu. W pracy współwystępują:

- pomiar niskopoziomowy,
- eksperyment pomostowy,
- benchmark realistyczny,
- walidacja poprawności,
- oraz interpretacja wsparta profilerem.

To sprawia, że rozprawa ma zarówno warstwę narzędziową, jak i wyjaśniającą. Nie jest tylko zbiorem eksperymentów, lecz uporządkowana metoda analizy zależności między architekturą, organizacją kernela i poprawnością obliczeń.

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

6.6. Jak czytać wyniki końcowe

Wyniki końcowe powinny być interpretowane w następującej kolejności:

1. czy backend przeszedł walidację poprawności,
2. jaki jest jego profil architektoniczny z mikrobenchmarków,
3. jak zachowują się próby walidacyjne FEM,
4. czy profiler wspiera te obserwacje,
5. dopiero na końcu - jaki jest czas wykonania i ranking konfiguracji.

Takie uporządkowanie chroni przed zbyt szybkim przejściem do prostego wniosku: "szybciej znaczy lepiej".

6.7. Konkluzja teoretyczna

W sensie teoretycznym v3 uzasadnia następujące podejście:

- wydajność należy analizować razem z poprawnością i z interpretacją mechanizmu,
- mikrobenchmarki zyskują wartość dopiero wtedy, gdy potrafią wyjaśnić zachowanie realistycznego kernela,
- a porównanie backendów staje się naukowo mocniejsze, gdy odbywa się na zamrożonych danych

Rozdział 6. Synteza wkładu i zakres wniosków

Podsumowanie wkładu własnego, ograniczeń i obszaru uzasadnionych wniosków.

wejściowych i z kontrola zgodności wyniku.

Pełna dokumentacja v3

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Plik źródłowy: docs/V3_READY_REFERENCE.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Ten dokument jest głównym, praktycznym przewodnikiem po v3.

Jesli chcesz:

- uruchamiać eksperymenty,
- sprawdzać gdzie zapisaly sie wyniki i wykresy,
- rozumiec jak lacza sie warstwy systemu,
- i miec jeden dokument startowy do codziennej pracy,

to zacznij od tego pliku.

1. Czym jest v3

v3 to uporządkowana platforma badawcza, w której:

- mikrobenchmarki mierzą cechy architektury,
- Walidacja opcji FEM sprawdza wzorce obliczeniowo-pamięciowe bliskie realistycznemu kernelowi FEM,
- referencyjny exact odtwarza referencyjny tor obliczeń,
- replay poprawności pozwala uruchomić te same zamrożone dane wejściowe na innym backendzie,
- korelacja profilerowa łączy wyniki mikrobenchmarków i walidacji z zachowaniem realistycznego kernela.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Najważniejsza narracja v3:

- mikrobenchmarki są główną warstwą badawczą,
- realistyczny kernel służy jako walidacja interpretacyjna,
- replay poprawności służy jako walidacja obliczeń,
- korelacja profilerowa służy jako warstwa wyjaśniająca.

2. Najważniejsze przepływy pracy

cpu_benchmark / gpu_benchmark

Warstwa mikrobenchmarków architektury.

Używaj jej do:

- pomiaru bandwidth,
- pomiaru latency,
- pomiaru compute throughput,
- budowy ceiling/roofline.

fem_option_validation

Twoja warstwa walidacji FEM bliska realistycznemu kernelowi.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Używaj jej do:

- sprawdzania wrażliwości na wzorce odczytu/zapisu,
- sprawdzania reuse/workspace/computation tradeoff,
- budowy mostu między mikrobenchmarkami a realistycznym kernelem.

filip_original

Warstwa referencyjna i realistyczna.

Zawiera:

- native performance campaign,
- referencyjny exact,
- replay poprawności.

profiler_correlation

Warstwa korelacyjna łącząca:

- najlepsze konfiguracje z kampanii,
- wyniki fem_option_validation,
- raporty profilerów.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

3. Jak uruchomic system

Start GUI

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python run_autotune_gui.py
```

```
```
```

Z GUI najważniejsze scenariusze

A. Mikrobenchmarki

Uruchamiaj przez przepływy pracy CPU/GPU.

B. Walidacja opcji FEM

W GUI:

- wybierz Workflow = Walidacja opcji FEM,
- ustaw backend,
- uruchom.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Artefakty pojawia się w:

- data/fem_option_validation/<run>/

Najważniejsze pliki:

- summary.json
- fem_option_validation.csv
- probe_summary.csv
- category_summary.csv
- fem_option_validation.md
- fem_option_validation_probes.png
- fem_option_validation_matrix.png

C. Reference exact / replay poprawności

W GUI:

- wybierz Workflow = Filip original,
- dla referencji OpenCL ustaw Filip mode = exact_reference,
- dla replayu na Metalu ustaw Replay dump root.

D. Korelacja profilerowa

W GUI:

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

- wybierz Workflow = Korelacja profilerowa,
- wskaz katalog optimization,
- wskaz katalog Walidacja opcji FEM,
- uruchom.

Artefakty pojawia sie w:

- <optimization_run>/profiler_correlation/

Najwazniejsze pliki:

- summary.json
- profiler_correlation.json
- option_alignment.csv
- profile_proximity.csv
- category_summary.csv
- profiler_correlation.md
- option_alignment.png
- profile_proximity.png
- category_summary.png

4. Gdzie sa wykresy

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

To jest ważne, bo to był realny problem roboczy.

Optimization / article-style runs

Wykresy zapisują się do:

- data/optimization/<run>/plots/

Najważniejsze wykresy czasu wykonania i ustawień autotuningu:

- article_filip_execution_time_by_option.png
- article_autotuning_trace_with_settings.png
- article_autotuning_settings_heatmap.png
- article_best_configuration_card.png

Ich znaczenie opisuje:

- docs/FILIP_TIMING_PLOTS.md

Walidacja opcji FEM

Wykresy zapisują się do:

- data/fem_option_validation/<run>/

Pliki:

- fem_option_validation_probes.png

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

- fem_option_validation_matrix.png

Korelacja profilerowa

Wykresy zapisują się do:

- data/optimization/<run>/profiler_correlation/

Pliki:

- option_alignment.png
- profile_proximity.png
- category_summary.png

Jak GUI wybiera wykres

GUI próbuje najpierw:

1. użyć zapisanych PNG z summary.json,
2. jeśli ich nie ma, narysować wykres dynamicznie z danych.

To oznacza:

- nowe runy po poprawce powinny mieć stałe artefakty PNG,
- starsze runy bez PNG nadal mogą być rysowane dynamicznie, o ile zawierają dane.

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

5. PDF-y dokumentacji

System generuje trzy glowne PDF-y:

docs/v3_teoria_od_podstaw.pdf

Teoretyczne wprowadzenie:

- dla osoby spoza informatyki,
- do spokojnego zrozumienia takich pojęć jak FEM, backend, replay, profiler i mikrobenchmark,
- jako material pomocniczy do pisania wstepu i metodologii.

docs/v3_ready_reference.pdf

Skondensowany dokument startowy:

- do codziennej pracy,
- do szybkiego onboardingu,
- do trzymania obok eksperymentow.

docs/v3_documentation_bundle.pdf

Pełny bundle dokumentacji:

- do archiwizacji,
- do pracy nad rozprawa,

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

- do przeglądu całego systemu.

Budowanie:

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python scripts/build_v3_documentation_pdf.py
```

```
```
```

6. Rekomendowana kolejność pracy

Jesli celem jest testowanie

1. README.md
2. docs/V3_READY_REFERENCE.md
3. docs/END_TO_END_TESTING_AND_WRITING.md
4. docs/EXPERIMENTAL_PROTOCOL.md

Jesli celem jest pisanie rozprawy

1. docs/TEORIA_OD_PODSTAW.md

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

2. docs/THESIS_RESEARCH_PLAN.md
3. docs/METHODOLOGY_MICROBENCH_TO_FEM.md
4. docs/THESIS_NAMING_MAP.md
5. docs/THREATS_TO_VALIDITY.md
6. docs/SYSTEM_REFERENCE.md

7. Co jest juz gotowe

Na poziomie platformy:

- uporządkowane przepływy pracy,
- rozdzielenie warstw badawczych,
- replay poprawności,
- provenance i hashe,
- kompaktowy pakiet replays,
- canonical bundles,
- zapisywane wykresy dla validation i correlation,
- GUI z loaderami wynikow i wykresow,
- komplet dokumentacji technicznej i metodologicznej.

8. Co sprawdzic przed finalnymi kampaniami

Przewodnik startowy

Praktyczny przewodnik do eksperymentów, wykresów i rozprawy.

Przed seria do rozprawy zrob:

1. smoke test fem_option_validation,
2. smoke test filip_original,
3. smoke test profiler_correlation,
4. potwierdzenie, ze summary.json zawiera plots,
5. potwierdzenie, ze PNG-i zapisały się w katalogu wyniku,
6. zapis wersji systemu i środowiska.

9. Jednozdaniowe podsumowanie

- v3 jest gotowa do realnych testów i do budowania rozprawy, o ile finalne kampanie będą prowadzone już na zamrozonym protokole eksperymentalnym.

Pełna dokumentacja v3

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

Plik źródłowy: docs/V3_DOCUMENTATION_INDEX.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

Ten indeks porządkuje dokumentację v3 w dwóch perspektywach:

- technicznej: jak uruchamiać, interpretować i archiwizować eksperymenty,
- doktoratowej: jak z tych eksperymentów zbudować obronną narrację naukową.

1. Dokumenty startowe

Jeśli wracasz do projektu po przerwie, zacznij od:

1. docs/V3_READY_REFERENCE.md
2. docs/PROJECT_MAP.md
3. docs/METHODOLOGY_MICROBENCH_TO_FEM.md
4. docs/END_TO_END_TESTING_AND_WRITING.md

1a. Gotowe PDF-y

Najważniejsze gotowe PDF-y to:

docs/v3_ready_reference.pdf

Skondensowany przewodnik startowy:

- do uruchamiania,

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

- do codziennej pracy,
- do szybkiej orientacji.

docs/v3_teoria_od_podstaw.pdf

Teoretyczne wprowadzenie:

- dla osoby spoza informatyki,
- do zrozumienia pojec takich jak FEM, backend, replay, profiler i mikrobenchmark,
- jako material pomocniczy do pisania wstepu i czesci metodologicznej.

docs/v3_documentation_bundle.pdf

Pełny bundle dokumentacji:

- technicznej,
- metodologicznej,
- eksperymentalnej,
- pisarskiej.

docs/v3_rozdzialy_teoretyczne.pdf

Zwarty pakiet rozdzialow teoretycznych:

- do czytania liniowego,
- do przerabiania na tekst rozprawy,
- do pracy nad wstepem, metodologia i zakresem wnioskow.

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

Budowanie PDF-ow:

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python scripts/build_v3_documentation_pdf.py
```

```
```
```

2. Dokumenty metodologiczne pod rozprawę

docs/TEORIA_OD_PODSTAW.md

Dokument wyjaśniający od zera:

- czym są obliczenia numeryczne,
- czym jest FEM,
- czym różnią się CPU, GPU i backend,
- po co są mikrobenchmarki,
- po co jest replay poprawności,
- dlaczego sam czas nie wystarcza.

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

docs/ROZDZIAL_01_MOTYWACJA_I_KONTEKST.md

Teoretyczny rozdział o problemie badawczym, motywacji i miejscu v3 w logice rozprawy.

docs/ROZDZIAL_02_OBLICZENIA_NUMERYCZNE_I_FEM.md

Rozdział wyjaśniający podstawy obliczeń numerycznych i metode elementow skonczonych.

docs/ROZDZIAL_03_ARCHITEKTURA_I_BACKENDY.md

Rozdział o CPU, GPU, pamieci, backendach i organizacji wykonania kernela.

docs/ROZDZIAL_04_METODOLOGIA_BADAWCZA_I_WALIDACJA.md

Rozdział metodologiczny opisujący mikrobenchmarki, walidacje FEM, exact i replay poprawnosci.

docs/ROZDZIAL_05_METRYKI_PROFILING_I_INTERPRETACJA.md

Rozdział o metrykach, profilerach, stabilnosci wynikow i interpretacji danych pomiarowych.

docs/ROZDZIAL_06_SYNTEZA_WKLADU_I_ZAKRES_WNIOSKOW.md

Rozdział spinający wkład własny, ograniczenia i zakres wnioskow rozprawy.

docs/V3_READY_REFERENCE.md

Najkrotszy dokument roboczy:

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

- czym jest v3,
- jak uruchamiać przepływy pracy,
- gdzie są wyniki i wykresy,
- jakie są dwa główne PDF-y,
- co sprawdzić przed finalnymi kampaniami.

docs/METHODOLOGY_MICROBENCH_TO_FEM.md

Główny dokument metodologiczny:

- rola mikrobenchmarków,
- rola reference_exact,
- rola correctness_replay,
- rola fem_option_validation,
- rola profiler_correlation.

docs/THESIS_NAMING_MAP.md

Nazewnictwo do rozprawy, prezentacji i publikacji:

- jak nazywać warstwy eksperymentalne,
- czego unikać,
- jak nie mieszać Twojego wkładu z benchmarkiem referencyjnym.

docs/THESIS_RESEARCH_PLAN.md

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

Plan naukowy rozprawy:

- główna teza,
- pytania badawcze,
- hipotezy,
- oczekiwany wkład,
- mapowanie pytań badawczych na przepływy pracy i artefakty.

docs/EXPERIMENTAL_PROTOCOL.md

Pełny protokół eksperymentalny:

- definicja kampanii,
- liczba powtorzeń,
- statystyka,
- polityka warmup / replay / profilerów,
- zasady archiwizacji,
- kryteria "paper-ready".

docs/THREATS_TO_VALIDITY.md

Zagrożenia dla trafności i ich ograniczenia:

- construct validity,
- internal validity,
- external validity,

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

- conclusion validity.

3. Dokumenty techniczne systemu

docs/PROJECT_MAP.md

Przegląd całego v3:

- główne warstwy,
- przepływy pracy,
- katalogi wyników,
- pakiet replays,
- provenance,
- GUI i entrypointy.

docs/SYSTEM_REFERENCE.md

Techniczny opis tego, co jest zaimplementowane:

- główne skrypty,
- role modułów,
- schemat przepływu danych,
- klasy eksperymentów,
- struktura wyników i artefaktów.

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

docs/CSV_SCHEMA.md

Kanoniczne pola CSV i ich znaczenie.

docs/METRICS.md

Definicje metryk:

- czas,
- throughput,
- GFLOP/s,
- energia,
- roofline.

docs/REPRODUCIBILITY.md

Uwagi dot. odtwarzalności kampanii.

4. Dokumenty uruchomieniowe

docs/UBUNTU_FILIP_SETUP.md

Linux/OpenCL:

- reference_exact,

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

- bundle export,
- replay preparation,
- fem_option_validation,
- profiler_correlation.

docs/FRESH_UBUNTU_BOOTSTRAP.md

Bootstrap swiezego Ubuntu.

docs/FIREFLY_OPTIMIZER.md

Dokumentacja firefly optimization.

docs/FILIP_TIMING_PLOTS.md

Opis wykresow pokazujacych czas wykonania kodu Filipa oraz ustawienia autotuningu.

docs/END_TO_END_TESTING_AND_WRITING.md

Praktyczna checklista:

- smoke tests,
- pelne przepływy pracy,
- archiwizacja wynikow,
- materialy do rozprawy.

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

5. Dokumenty starsze lub pomocnicze

docs/MICROBENCH_DOKUMENTACJA.md

Szerszy opis warstwy mikrobenchmarkow.

docs/MICROBENCH_DOKUMENTACJA.pdf

Wersja PDF powyzszej dokumentacji.

6. Rekomendowana kolejnosc czytania pod doktorat

Jesli celem jest pisanie rozprawy, rekomendowana kolejnosc jest taka:

1. docs/TEORIA_OD_PODSTAW.md
2. docs/V3_READY_REFERENCE.md
3. docs/THESIS_RESEARCH_PLAN.md
4. docs/METHODOLOGY_MICROBENCH_TO_FEM.md
5. docs/THESIS_NAMING_MAP.md
6. docs/EXPERIMENTAL_PROTOCOL.md
7. docs/THREATS_TO_VALIDITY.md

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

8. docs/END_TO_END_TESTING_AND_WRITING.md

9. docs/SYSTEM_REFERENCE.md

7. Rekomendowana kolejność czytania pod implementację

Jeśli celem jest rozwijanie lub uruchamianie systemu:

1. docs/V3_READY_REFERENCE.md

2. docs/PROJECT_MAP.md

3. docs/SYSTEM_REFERENCE.md

4. docs/UBUNTU_FILIP_SETUP.md

5. docs/END_TO_END_TESTING_AND_WRITING.md

6. docs/CSV_SCHEMA.md

7. docs/METRICS.md

8. docs/REPRODUCIBILITY.md

8. Jednozdaniowe podsumowanie v3

Najważniejsza narracja v3 brzmi:

Indeks dokumentacji

Mapa dokumentów i zależności między nimi.

- v3 to platforma, która łączy mikrobenchmarki architektury z realistycznym kernelem FEM przez warstwy prób walidacyjnych, replay poprawności na zamrożonych danych wejściowych i korelację profilerowa.

Pełna dokumentacja v3

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Plik źródłowy: docs/PROJECT_MAP.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Repo w wersji v3 jest zorganizowane wokół czterech warstw eksperymentalnych:

1. Mikrobenchmarki
2. Referencyjne kernele exact
3. Replay poprawności
4. Natywne kampanie aplikacyjne

Taki podział jest celowy. Mikrobenchmarki są główną warstwą badawczą. Exact/replay służą do walidacji poprawności i do sprawdzania, czy wnioski architektoniczne z mikrobenchmarków tłumaczą zachowanie realistycznego kernela FEM.

Najważniejsze dokumenty metodologiczne:

- docs/V3_DOCUMENTATION_INDEX.md
- docs/METHODOLOGY_MICROBENCH_TO_FEM.md
- docs/THESIS_RESEARCH_PLAN.md
- docs/THESIS_NAMING_MAP.md
- docs/EXPERIMENTAL_PROTOCOL.md
- docs/THREATS_TO_VALIDITY.md
- docs/SYSTEM_REFERENCE.md
- docs/END_TO_END_TESTING_AND_WRITING.md

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

1. CPU mikrobenchmarki

Katalogi:

- `cpu/lib/`
- `cpu/benchmarks/`
- `run_all_cpu_benchmarks.py`

Główne benchmarki:

- `run_bandwidth.py`
- `run_bandwidth_mt.py`
- `run_pointer_latency.py`
- `run_compute_fma.py`
- `run_compute_fma_peak.py`

Wyniki trafiają zwykle do `data/cpu/`.

2. GPU mikrobenchmarki

Katalogi:

- `gpu/metal/`

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

- gpu/cuda/
- gpu/hip/
- gpu/opencv/
- run_all_gpu_benchmarks.py

Backendy:

- metal - macOS / Apple GPU
- cuda - Linux / NVIDIA
- hip - Linux / AMD ROCm
- opencv - przekrojowo dla Intel/AMD/NVIDIA

Te benchmarki sa podstawa porownan architektonicznych. To tutaj mierzone sa cechy sprzetu, ktore pozniej interpretuja zachowanie kerneli FEM.

3. Filip referencyjny exact

Głowne pliki:

- run_filip_reference_exact.py
- Kod Filipa/mod_2022/
- opcjonalnie legacy/filip_exact_bundle/mod_2022/

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Ta warstwa służy do uruchomienia możliwie wiernej referencji legacy OpenCL Filipa.

Klasy eksperymentów w `summary.json`:

- `reference_exact`
- `correctness_replay`
- `native_performance_campaign`

Najważniejsze znaczenia:

- `reference_exact`
 - Linux/OpenCL
 - oryginalny tor Filipa
 - źródło prawdy dla wejść i wyników
- `correctness_replay`
 - Metal replay na zamrożonych wejściach OpenCL
 - walidacja poprawności 1:1 w granicach tolerancji
- `native_performance_campaign`
 - natywna kampania wydajnościowa projektu
 - nie jest ściśle 1:1 z legacy OpenCL

4. Replay bundles

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Nowe skrypty v3:

- scripts/export_filip_replay_inputs.py
- scripts/export_canonical_replay_bundles.py

Compact pakiet replay

To mały artefakt, który zawiera tylko dane niezbędne do odtworzenia tego samego wejścia kernela na innym backendzie:

- launch_meta.json
- execution_parameters.bin
- gauss_dat.bin
- shape_fun_ref.bin
- el_data_in.bin
- opcjonalnie el_data_out.bin

Ten bundle jest głównym artefaktem do walidacji poprawności.

Canonical pakiet replays

To małe, stałe zestawy regresyjne, przeznaczone do szybkich testów poprawności i utrzymania translatora replay.

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Aktualne presety:

- smoke_test_prism_qss_opt000
- smoke_test_prism_sqs_opt017
- smoke_laplace_prism_ssqr_opt062

5. Walidacja opcji FEM

Nowy przepływ pracy v3:

- run_fem_option_validation.py

Cel:

- zmierzyc w kontrolowany sposób wpływ opcji walidacyjnych na wykonanie
- zachowac multiplatformowosc
- powiazac przestrzen opcji walidacyjnych z cechami architektury

To nie jest replay legacy kernela. To jest kontrolowany zestaw prob lokalnych, dzialajacy przez wspolny FemParametricProblem.

Aktualne grupy prob:

- coal_read

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

- coal_write
- compute_all_shape_fun_der
- workspace_pde_coeff
- workspace_geo
- workspace_shape_fun
- workspace_stiff_mat
- padding

Wyniki trafiają do data/fem_option_validation/.

6. Korelacja profilerowa

Nowy przepływ pracy v3:

- scripts/run_profiler_correlation.py

Cel:

- zestawic:
 - wynik exact/native kampanii FEM
 - wynik Walidacja opcji FEM
 - opcjonalne eksporty z profilerow
- zapisac jeden raport korelacyjny

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Wyjście:

- profiler_correlation.json
- profiler_correlation.md

To jest warstwa interpretacyjna, łącząca mikrobenchmarki, profilery i realistyczny kernel.

7. Dekodowanie el_data_out

Nowy modul v3:

- analysis/filip_output_decode.py

Cel:

- zamienić płaski el_data_out.bin na bardziej czytelne artefakty:
 - stiffness_matrixelem000.csv
 - rhs_vectorelem000.csv
 - decoded_output_summary.json

Dzisiaj dekodery są przygotowane głównie pod skalarne przypadki exact ($n_{req} = 1$). To wystarcza dla obecnych laplace_prism i test_prism.

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

8. Provenance i reproducibility

Nowy modul v3:

- analysis/provenance.py

Co zapisujemy:

- hash plików
- hash summary/bundle
- runtime provenance
- informacje Git, jeśli repo jest dostępne

To wzmacnia reprodukowalność i pozwala traktować bundle oraz wyniki jako artefakty badawcze, a nie tylko pomocnicze pliki robocze.

9. Workflow orchestration i GUI

Główne entrypointy:

- run_przepływ_pracy.py
- run_autotune_gui.py

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

Nowe przepływy pracy v3:

- filip_original
- fem_option_validation
- profiler_correlation
- filip_autotune
- filip_firefly

GUI pozwala teraz:

- uruchomic exact OpenCL
- wyeksportowac kompaktowy pakiet replay
- wyeksportowac kanoniczne pakiety replay
- uruchomic Metal replay z Replay dump root
- uruchomic Walidacja opcji FEM
- uruchomic korelacja profilerowa
- podejrzec zapisany output i jego CSV
- zobaczyc zdekodowana macierz i RHS w summary exact runu

10. Jak czytac repo metodologicznie

Najwazniejsze rozroznienie:

- mikrobenchmarki odpowiadaja na pytanie: "jak zachowuje sie architektura?"

Mapa projektu

Struktura repozytorium i główne warstwy eksperymentalne.

- `reference_exact` odpowiada na pytanie: "jak wygląda referencyjne wykonanie legacy kernela?"
- `correctness_replay` odpowiada na pytanie: "czy inny backend liczy to samo na tych samych danych?"
- `native_performance_campaign` odpowiada na pytanie: "jak zachowuje się natywna implementacja projektu?"

To rozroznienie jest kluczowe dla interpretacji wyników w doktoracie.

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

Plik źródłowy: docs/SYSTEM_REFERENCE.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

Ten dokument jest technicznym opisem tego, co jest zaimplementowane w v3.

1. Główna idea systemu

v3 nie jest pojedynczym benchmarkiem, tylko platforma eksperymentalna złożona z warstw:

1. mikrobenchmarki
2. fem_option_validation
3. reference_exact
4. correctness_replay
5. native_performance_campaign
6. profiler_correlation

Każda warstwa ma inną rolę:

- mikrobenchmarki opisują architekturę,
- fem_option_validation buduje warstwę interpretacyjną blisko realistycznego kernela,
- reference_exact daje źródło zamrożone dane wejściowe,
- correctness_replay sprawdza równoważność wyniku,
- native_performance_campaign bada natywne backendy projektu,
- profiler_correlation scala warstwy interpretacyjne.

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

2. Główne entrypointy

GUI

- run_autotune_gui.py

Orkiestracja przepływów pracy

- run_przepływ_pracy.py

Kampanie mikrobenchmarkowe

- run_all_cpu_benchmarks.py
- run_all_gpu_benchmarks.py
- run_all_benchmarks.py
- run_all_backends.py

Kampanie realistyczne / FEM

- run_filip_original.py
- run_filip_reference_exact.py
- run_fem_option_validation.py
- scripts/run_profiler_correlation.py

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

3. Klasy eksperymentów

W summary.json i w narracji v3 występują trzy podstawowe klasy w obszarze FEM:

reference_exact

- uruchomienie legacy OpenCL exact
- Linux / OpenCL
- źródło zamrożone dane wejściowe i oczekiwane wyniki

correctness_replay

- replay na zamrożone dane wejściowe
- np. Metal replay na wejściach OpenCL
- walidacja outputu

native_performance_campaign

- natywna kampania wydajnościowa projektu
- nie jest strict replayem 1:1

Dodatkowo:

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

fem_option_validation

- kontrolowane probe'y walidacyjne

profiler_correlation

- raport łączący probe'y, wynik FEM i profiler

4. Główna struktura katalogów

Kod

- cpu/
- gpu/
- real_kernels/
- analysis/
- scripts/
- optimization/
- Kod Filipa/
- legacy/

Wyniki

- data/cpu/

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- data/gpu/
- data/optimization/
- data/fem_option_validation/
- data/runs/
- reports/

5. Warstwa fem_option_validation

Entry point

- run_fem_option_validation.py

Cel

- zbadać lokalny wpływ kontrol odpowiadających:
 - coal_read
 - coal_write
 - compute_all_shape_fun_der
 - workspace_*
 - padding

Architektura

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- oparta na FemParametricProblem
- wykonywana przez wspólny cross-platform problem layer
- nie jest strict replayem legacy exact

Artefakty

- fem_option_validation.csv
- records.jsonl
- probe_summary.csv
- category_summary.csv
- fem_option_validation.md
- summary.json

Najważniejsze pola summary

- przepływ pracy
- experiment_class
- backend
- execution_mode
- device
- records
- probe_catalog
- probe_summary

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- category_summary
- provenance
- summary_hash

6. Warstwa profiler_correlation

Entry point

- scripts/run_profiler_correlation.py

Cel

- powiazac:
 - wynik exact/native,
 - wynik fem_option_validation,
 - opcjonalne raporty profilerow.

Artefakty

- profiler_correlation.json
- summary.json
- profiler_correlation.md
- option_alignment.csv

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- profile_proximity.csv
- category_summary.csv

Najważniejsze pola summary

- przepływ pracy
- optimization_dir
- fem_option_validation_dir
- best_overall
- option_alignment
- profile_proximity
- category_summary
- profiler_reports
- provenance
- summary_hash

7. Warstwa exact / replay

Entry point

- run_filip_reference_exact.py

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

Role

- reference_exact
- correctness_replay
- native_performance_campaign

Exact OpenCL

- uruchamiany na Linuxie
- używa legacy mod_2022
- potrafi budować exact kernel path
- zapisuje launch dumps i wyniki numeryczne

Replay

- wykorzystuje zamrożone dane wejściowe z exact
- potrafi odpalić Metal replay
- waliduje output, jeśli jest expected output

Dodatkowe artefakty v3

- kompaktowy pakiet replay
- kanoniczne pakiety replay
- decoded matrix / rhs outputs

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- provenance i hashe

8. Replay bundles

Compact pakiet replay

Zawiera:

- launch_meta.json
- execution_parameters.bin
- gauss_dat.bin
- shape_fun_ref.bin
- el_data_in.bin
- opcjonalnie el_data_out.bin

Canonical pakiet replays

Małe zestawy regresyjne do:

- szybkich testów,
- walidacji replayu,
- utrzymania translatora.

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

9. Dekoder outputu

Modul

- analysis/filip_output_decode.py

Cel

- przetworzyc el_data_out.bin do:
 - stiffness_matrixelem000.csv
 - rhs_vectorelem000.csv
 - decoded_output_summary.json

Stan

- przygotowany glownie dla przypadkow skalarowych exact ($n_{req} = 1$)

10. Provenance

Modul

- analysis/provenance.py

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

Co robi

- zbiera informacje runtime,
- zapisuje srodowisko,
- liczy hashe plikow,
- zapisuje summary_hash,
- pobiera informacje Git, jesli repo jest dostepne.

Gdzie jest uzywany

- fem_option_validation
- profiler_correlation
- exact / replay

11. GUI

Glowny plik

- run_autotune_gui.py

Najwazniejsze funkcje

- uruchamianie przepływów pracy
- live output

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- live utilization
- ładowanie wyników
- probe support panel
- ładowanie exact output CSV
- ładowanie latest validation / correlation
- wykresy dla:
 - optimization
 - validation
 - correlation
 - matrix comparison

Ważne przyciski v3

- Load Latest FEM Validation
- Load Latest Correlation
- Load Summary
- Loaded Result Plot
- Validation Plot
- Correlation Plot

12. Orkiestracja przepływów pracy

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

Plik

- run_przepływ pracy.py

Odpowiedzialność

- zamiana przyjaznych przepływów pracy na konkretne skrypty i argumenty,
- wykrywanie backendów,
- rozstrzyganie backend tokens (intel, amd, auto),
- spójne zwracanie out_dir i summary.

Najważniejsze przepływy pracy v3

- cpu_benchmark
- gpu_benchmark
- cpu_real_kernels
- gpu_real_kernels
- filip_original
- fem_option_validation
- profiler_correlation
- filip_autotune
- filip_firefly

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

13. Artefakty pod rozprawę

Najważniejsze artefakty, które system już produkuje:

Dla validation

- probe_summary.csv
- category_summary.csv
- fem_option_validation.md

Dla correlation

- option_alignment.csv
- profile_proximity.csv
- profiler_correlation.md

Dla exact/replay

- validation_summary
- pakiet replays
- decoded outputs

14. Co system już robi dobrze

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

1. rozdziela correctness od performance,
2. wspiera replay na zamrożonych danych wejściowych,
3. ma warstwe probe'ow walidacyjnych,
4. wspiera provenance i hash-based reproducibility,
5. daje GUI i CLI dla całego pipeline'u,
6. zachowuje multiplatformowosc.

15. Co nie jest jeszcze uniwersalne

Należy to traktować jako ograniczenia systemu, nie jako błąd dokumentacji:

- replay translator nie jest ogólnym kompilatorem OpenCL -> Metal,
- dekodery outputu są dziś najmocniejsze dla $n_{req} = 1$,
- korelacja profilerowa wymaga sensownych eksportów z zewnętrznych profilerów,
- native_performance_campaign nie jest strict exact replayem.

16. Jak korzystać z tej dokumentacji

Jeśli chcesz:

Referencja systemowa

Opis techniczny zaimplementowanych workflowów i artefaktów.

- zrozumiec metodyke: czytaj METHODOLOGY_MICROBENCH_TO_FEM.md
- planowac doktorat: czytaj THESIS_RESEARCH_PLAN.md
- prowadzic eksperymenty: czytaj EXPERIMENTAL_PROTOCOL.md
- uruchamiac system: czytaj END_TO_END_TESTING_AND_WRITING.md

Metodologia: od mikrobenchmarków do

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Plik źródłowy: docs/METHODOLOGY_MICROBENCH_TO_FEM.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Ten dokument porządkuje metodologie v3 pod kątem doktoratu.

1. Główna teza metodologiczna

Podstawa pracy to mikrobenchmarki, a nie benchmark Filipa jako taki.

Rola warstw jest następująca:

- mikrobenchmarki opisują cechy architektury
- benchmark Filipa dostarcza realistycznego kernela numerycznego
- replay OpenCL -> Metal sprawdza, czy porównanie dotyczy tego samego obliczenia
- korelacja profilerowa łączy przewidywania z mikrobenchmarków z rzeczywistym zachowaniem kernela

To oznacza, że benchmark Filipa nie jest głównym wkładem sam w sobie. Jest walidatorem aplikacyjnym i referencją obliczeniową.

2. Cztery klasy eksperymentów

A. Mikrobenchmarki

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Pytanie badawcze:

- jakie cechy architektury ograniczają wydajność?

Typowe metryki:

- bandwidth
- pointer latency
- FMA throughput
- peak compute
- sensitivity na workgroup size i wzorzec pamięci

B. Reference exact

Pytanie badawcze:

- jak wygląda referencyjne wykonanie legacy kernela Filipa?

To jest źródło prawdy dla:

- zamrożone dane wejściowe
- oczekiwane wyniki
- exact campaign metadata

C. Replay poprawności

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Pytanie badawcze:

- czy inny backend liczy to samo na tych samych danych?

To jest warstwa walidacji poprawności. Najważniejsze kryterium nie brzmi "czy czas jest podobny", tylko:

- czy output dla tego samego frozen input zgadza się w granicach tolerancji

D. Native performance campaign

Pytanie badawcze:

- jak zachowuje się natywna implementacja projektu?

Ta warstwa jest wydajnościowo bardzo ważna, ale nie wolno jej mylic z strict replay poprawności.

3. Dlaczego sam mesh i plik problemu nie wystarczają

Dla walidacji 1:1 nie wystarczy tylko:

- mesh_prism.dmp
- problem_conv_diff.dat

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

To są dane wysokiego poziomu. OpenCL kernel legacy liczy już na zbudowanych buforach runtime:

- execution_parameters.bin
- gauss_dat.bin
- shape_fun_ref.bin
- el_data_in.bin

Dopiero zamrożenie tych buforów gwarantuje, że dwa backendy dostają naprawdę to samo wejście do obliczenia.

4. Rola kompaktowy pakiet replay

Compact pakiet replay to główny artefakt walidacyjny v3.

Zawiera:

- launch_meta.json
- execution_parameters.bin
- gauss_dat.bin
- shape_fun_ref.bin
- el_data_in.bin
- opcjonalnie el_data_out.bin

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Wartosc metodologiczna:

- minimalizuje koszt przenoszenia danych
- zamraza rzeczywiste wejście do kernela
- pozwala wykonać replay poprawności na innym backendzie
- może zawierać expected output do automatycznej walidacji

5. Rola Walidacja opcji FEM

Nowa warstwa `fem_option_validation` odpowiada bezpośrednio opcjom walidacyjnym istotnym dla kampanii exact, ale robi to w sposób kontrolowany i multiplatformowy.

To nie jest exact replay. To jest most interpretacyjny między:

- prostymi mikrobenchmarkami architektury
- a realistyczna przestrzenia opcji kernela FEM

Przykładowe osie:

- `coal_read`
- `coal_write`
- `compute_all_shape_fun_der`
- `workspace_*`
- `padding`

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Ta warstwa ma odpowiedzieć na pytanie:

- czy lokalny wpływ danej opcji jest zgodny z tym, czego spodziewamy się po danej architekturze?

Praktyczne artefakty tej warstwy:

- fem_option_validation.csv
- probe_summary.csv
- category_summary.csv
- fem_option_validation.md

6. Rola korelacja profilerowa

Korelacja profilerowa jest warstwa łącząca:

- wynik kampanii exact/native
- wynik Walidacja opcji FEM
- eksporty z profilerów

To pozwala przejść od prostego stwierdzenia:

- "opcja A była szybsza"

Do silniejszego stwierdzenia:

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

- "opcja A była szybsza, bo mikrobenchmarki wskazują ograniczenie pamięci, a profiler pokazuje zgodny wzorzec bandwidth/stalls"

Praktyczne artefakty tej warstwy:

- profiler_correlation.json
- profiler_correlation.md
- option_alignment.csv
- profile_proximity.csv
- category_summary.csv

7. Correctness vs performance

Te dwie rzeczy muszą być rozdzielone.

Correctness

Najsilniejsze kryterium:

- ten sam frozen input
- ten sam expected output
- $\max_abs_diff \leq tol$
- małe rms_diff

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Performance

Najsilniejsze kryterium:

- porównywanie backendów dopiero po upewnieniu się, że licza to samo
- analiza czasu, bandwidth, FLOPS i energii dopiero na tle potwierdzonej poprawności

8. Co można twierdzić naukowo

Mozna uczciwie twierdzić:

- mikrobenchmarki opisują cechy architektury
- te cechy mają wartość wyjaśniającą dla realistycznego kernela FEM
- replay poprawności pokazuje, że porównanie backendów dotyczy tego samego obliczenia

Nie należy twierdzić bez dodatkowych zastrzeżeń:

- że natywna kampania `native_performance_campaign` jest ściśle 1:1 z legacy exact
- że podobny czas oznacza automatycznie poprawność obliczeń

9. Co stanowi najmocniejszy wkład v3

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Najmocniejsze elementy v3 to:

- uporządkowana metodologia od mikrobenchmarku do realistycznego kernela
- correctness-aware replay na zamrożone dane wejściowe
- Walidacja opcji FEM odpowiadający opcjom walidacyjnym istotnym dla kampanii exact
- provenance i hash-based reproducibility
- korelacja profilerowa jako warstwa interpretacyjna

To jest silniejszy wkład niż sama reprodukcja benchmarku Filipa.

10. Co dalej warto dodawać

Najbardziej sensowne kolejne kroki:

- rozbudować Walidację opcji FEM o bardziej wyspecjalizowane wzorce pamięci i reuse
- rozwinąć dekodery `el_data_out` dla większej liczby przypadków i `nreq > 1`
- dodać bardziej formalne raporty statystyczne i przedziały niepewności
- dopiąć lepszy eksport z profilerów (VTune, Xcode GPU, Nsight, ROCm)
- przygotować małe kanoniczne bundle regresyjne jako stałe artefakty testowe

To wszystko wzmacnia główną rolę pracy: mikrobenchmarki jako narzędzie wyjaśniające, a nie tylko pomiarowe.

Metodologia: od mikrobenchmarków do FEM

Jak warstwa architektoniczna łączy się z realistycznym kernelem FEM.

Zobacz też:

- docs/THESIS_NAMING_MAP.md
- docs/END_TO_END_TESTING_AND_WRITING.md

Wykresy czasu wykonania i autotuning

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji

Plik źródłowy: docs/FILIP_TIMING_PLOTS.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

Ten dokument opisuje dodatkowe wykresy generowane dla kampanii związanych z realistycznym kernelem FEM i kodem referencyjnym Filipa.

1. Po co są te wykresy

Ich celem jest pokazanie nie tylko tego, która konfiguracja była najszybsza, ale także dlaczego była najszybsza z punktu widzenia ustawień autotuningu.

W praktyce łączy one trzy informacje:

- czas wykonania,
- wariant algorytmiczny, np. QSS, SQS, SSQ,
- ustawienia opcji, np. koalescencja odczytu, koalescencja zapisu, workspace, padding i sposób liczenia pochodnych funkcji kształtu.

Dzięki temu można użyć ich bezpośrednio w rozprawie do pokazania, że autotuning nie jest czarna skrzynka, tylko mierzalnym przeszukiwaniem konkretnych decyzji implementacyjnych.

2. Gdzie się zapisują

Dla runów w `data/optimization/<run>/` wykresy trafiają do:

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

```
```text
```

```
data/optimization/<run>/plots/
```

```
```
```

Nowe pliki to:

```
```text
```

```
article_filip_execution_time_by_option.png
```

```
article_autotuning_trace_with_settings.png
```

```
article_autotuning_settings_heatmap.png
```

```
article_best_configuration_card.png
```

```
```
```

GUI traktuje je jako preferowane wykresy dla Optimization Plot i Loaded Result Plot.

3. Znaczenie poszczególnych wykresów

article_filip_execution_time_by_option.png

Pokazuje czas wykonania dla opcji autotuningu.

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

Na osi poziomej są maski bitowe opcji, czyli skrócony zapis decyzji typu:

- czy włączono koalescencje odczytu,
- czy włączono koalescencje zapisu,
- czy użyto workspace,
- czy włączono padding,
- czy liczone wszystkie pochodne funkcji kształtu.

Na osi pionowej jest czas wykonania. Dla exact reference jest to czas oryginalnej ścieżki referencyjnej, a dla kampanii natywnych znormalizowany czas wykonania kernela FEM.

Wykres zaznacza najlepszą konfigurację i wypisuje najważniejsze aktywne ustawienia.

article_autotuning_trace_with_settings.png

Pokazuje przebieg autotuningu w czasie.

Wykres odpowiada na pytania:

- jak szybko autotuner znalazł dobrą konfigurację,
- czy najlepszy wynik pojawił się przypadkiem dopiero na końcu,
- czy kolejne ewaluacje rzeczywiście poprawiały wynik,

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

- który wariant algorytmiczny dominował w najlepszych punktach.

Punkty reprezentują kolejne ewaluacje, a linia najlepszy do tej pory pokazuje postęp optymalizacji.

article_autotuning_settings_heatmap.png

Pokazuje najlepsze konfiguracje i ich ustawienia.

Wiersze to najszybsze konfiguracje. Kolumny to opcje autotuningu:

- coal read,
- coal write,
- all shape der.,
- ws pde,
- ws geo,
- ws shape,
- ws stiff,
- padding.

Wartosci 0/1 oznaczają, czy dana opcja była wyłączona czy włączona. Po prawej stronie jest pasek z czasem wykonania.

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

Ten wykres jest szczególnie przydatny do odpowiedzi na pytanie: które decyzje implementacyjne powtarzają się wśród najlepszych konfiguracji.

article_best_configuration_card.png

To karta najlepszej konfiguracji.

Zawiera:

- backend i urządzenie,
- operator i wariant,
- maske bitowa opcji,
- rozmiar problemu,
- workgroup size,
- czas wykonania,
- GFLOP/s i GB/s,
- liste flag autotuningu.

To jest wykres-opis, dobry do szybkiego wstawienia do materiałów roboczych lub do appendixu.

4. Jak wygenerować ręcznie

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji.

Dla dowolnego runu:

```
```bash
```

```
cd
```

```
/Users/mateusznytko/Desktop/Praca/Doktorat/Testy/Testy_grudzien/apple_microbench_va
```

```
if [-f .venv/bin/activate]; then source .venv/bin/activate; fi
```

```
python analysis/filip_article_plots.py --optimization-dir data/optimization/<run>
```

```
```
```

5. Jak interpretować w rozprawie

Te wykresy nie mają zastępować mikrobenchmarków. Ich rola jest inna:

- mikrobenchmarki wyjaśniają cechy architektury,
- te wykresy pokazują, jak decyzje autotuningu przekładają się na czas realistycznego kernela,
- replay i exact reference potwierdzają, że porównanie dotyczy sensownego obliczenia.

Najbezpieczniejsze sformułowanie do rozprawy:

> Dodatkowe wykresy czasu wykonania łączą wynik pomiaru z konfiguracją autotuningu. Pozwala to analizować nie tylko najlepszą wartość czasu, ale również powtarzalność decyzji

Wykresy czasu wykonania i autotuningu

Jak interpretować wykresy czasu wykonania, ustawień autotuningu i najlepszych konfiguracji. implementacyjnych wśród najlepszych konfiguracji.

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

Plik źródłowy: docs/THESIS_RESEARCH_PLAN.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisanie rozprawy.

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

Ten dokument porządkuje część naukowa pracy doktorskiej wokół v3.

1. Główny cel pracy

Głównym celem pracy nie jest reprodukcja benchmarku referencyjnego sama w sobie, ale:

- zbudowanie metodologii, która łączy mikrobenchmarki architektury z zachowaniem realistycznego kernela FEM,
- oraz sprawdzenie, czy taka metodologia daje:
 - wartość wyjaśniającą,
 - poprawną walidację między backendami,
 - i podstawę do interpretacji wyników wydajnościowych.

2. Główna teza

Najsilniejsza wersja tezy doktorskiej:

- Metodologia łącząca mikrobenchmarki architektury, próby walidacyjne FEM, replay poprawności na zamrożonych danych wejściowych oraz korelacje profilerowe pozwala wiarygodniej wyjaśnić i walidować zależne od backendu zachowanie realistycznego kernela FEM niż sama analiza czasów

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

wykonania.

3. Pytania badawcze

RQ1. Cechy architektury

- Ktore cechy architektury ujawnione przez mikrobenchmarki sa najistotniejsze dla zachowania kernela FEM?

RQ2. Wartosc probe'ow walidacyjnych

- Czy fem_option_validation stanowi sensowny most interpretacyjny miedzy prostymi mikrobenchmarkami a realistyczna przestrzenia opcji kernela FEM?

RQ3. Poprawnosc porownan backendow

- Czy replay na zamrożonych danych wejściowych pozwala stwierdzic, ze dwa backendy licza to samo na tych samych danych?

RQ4. Wartosc profilerow

- Czy dane z profilerow wzmacniaja interpretacje wynikow wynikajaca z mikrobenchmarkow i probe'ow walidacyjnych?

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

RQ5. Uzytecznosc metodologii

- Czy cala warstwa mikrobenchmarki -> validation probes -> exact/replay -> korelacja profilerowa daje praktycznie uzyteczny i obronny przeplyw pracy badawczy dla wieloplatformowych backendow FEM?

4. Hipotezy badawcze

H1. Mikrobenchmarki maja wartosc wyjasniajaca

- Peak bandwidth, pointer latency, compute throughput i sensitivity na layout/workgroup size beda istotnie skorelowane z rankingiem konfiguracji w realistycznym kernelu FEM.

H2. Walidacja opcji FEM jest wartosciowym mostem

- Probe'y walidacyjne ukierunkowane na coal_read, coal_write, compute_all_shape_fun_der, workspace_* i padding beda zgodne kierunkowo z najlepszymi konfiguracjami w kampanii exact/native.

H3. Frozen-input replay jest warunkiem uczciwego porownania

- Porownanie backendow bez replay na zamrozonych danych wejsciowych moze dawac mylacy obraz poprawnosci, podczas gdy replay na tych samych danych umozliwi walidacje wyjscia w granicach tolerancji.

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

H4. Profiler-assisted correlation wzmacnia interpretacje

- Liczniki profilerowe beda zgodne z interpretacja wynikow wynikajaca z mikrobenchmarkow i probe'ow walidacyjnych dla dominujacych bottleneckow.

H5. Sama analiza czasu nie wystarcza

- Same czasy wykonania nie beda wystarczajace do poprawnej interpretacji roznic miedzy backendami bez warstwy poprawnosci i interpretacji architektonicznej.

5. Wklad własny pracy

Najmocniejsze elementy wkładu:

1. Metodologia wyjasniajaca:

- polaczenie mikrobenchmarkow z realistycznym kernelem FEM.

2. Walidacja opcji FEM:

- multiplatformowa warstwa probe'ow odpowiadajaca kluczowym kontrolom istotnym dla kernela FEM.

3. Frozen-input replay poprawności:

- porownywanie backendow na tych samych danych wejściowych do obliczen.

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

4. Profiler-assisted correlation:

- warstwa spinajaca wyniki mikrobenchmarkow, probe'ow i realistycznego kernela.

5. Reproducibility / provenance:

- hash-based artefakty i uporządkowany przepływ pracy eksperymentalny.

6. Co nie jest glownym wkladem

To jest bardzo wazne i warto to napisac wprost.

Nie nalezy przedstawiac jako glowny wklad:

- samego benchmarku referencyjnego,
- samej reprodukcji legacy OpenCL,
- samego uruchomienia kernela na innym backendzie bez warstwy poprawnosci i interpretacji.

Benchmark referencyjny pelni role:

- aplikacyjnej referencji,
- zrodla zamrożone dane wejściowe,
- zewnętrznego walidatora hipotez wynikających z mikrobenchmarkow.

7. Mapa pytan badawczych na przepływy pracy

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

RQ1 / H1

Uzyj:

- cpu_benchmark
- gpu_benchmark
- roofline
- METRICS.md

Artefakty:

- session summaries,
- roofline outputs,
- benchmark CSV.

RQ2 / H2

Uzyj:

- fem_option_validation

Artefakty:

- fem_option_validation.csv
- probe_summary.csv
- category_summary.csv

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

- fem_option_validation.md

RQ3 / H3

Uzyj:

- reference_exact
- correctness_replay
- compact/kanoniczne pakiety replay

Artefakty:

- launch_dumps/
- replay_inputs_bundle/
- validation_summary
- decoded outputs

RQ4 / H4

Uzyj:

- profiler_correlation
- eksporty profilerow

Artefakty:

- profiler_correlation.md

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

- option_alignment.csv
- profile_proximity.csv
- category_summary.csv

RQ5 / H5

Uzyj calego pipeline'u:

- mikrobenchmarki
- validation probes
- exact/replay
- korelacja profilerowa

Artefakty:

- przekrojowe tabele i wykresy z wielu backendow.

8. Minimalny korpus eksperymentalny do doktoratu

Potrzebujesz co najmniej:

1. Jednej architektury CPU
2. Jednej architektury Apple GPU / Metal
3. Jednej architektury Intel GPU / OpenCL

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

4. Opcjonalnie:

- NVIDIA / CUDA
- AMD / HIP

Na kazdej architekturze:

- mikrobenchmarki,
- fem_option_validation,
- przynajmniej jedna realistyczna kampania FEM,
- korelacja profilerowa,
- replay poprawnosci dla reprezentatywnych przypadkow.

9. Minimalne kryteria sukcesu dla tez

Dla H1

- widoczna zgodnosc miedzy peak/sensitivity z mikrobenchmarkow a rankingiem opcji w realistycznym kernelu.

Dla H2

- option_alignment i profile_proximity pokazuja sensowna zgodnosc probe'ow z najlepszymi konfiguracjami.

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

Dla H3

- replay daje zgodnosc wynikow wyjsciowych w granicach tolerancji.

Dla H4

- przynajmniej czesc profilerowych bottleneckow jest zgodna z interpretacja mikrobenchmarkowa.

Dla H5

- mozna pokazac przypadki, gdzie sama analiza czasu bylaby mylaca, a warstwa poprawnosci i korelacji usuwa niejednoznaczosc.

10. Proponowany uklad rozdzialow

1. Charakterystyka architektury z uzyciem mikrobenchmarkow
2. Proby walidacyjne opcji FEM jako warstwa interpretacyjna
3. Referencyjna kampania exact i replay poprawnosci na zamrozonych danych wejsciowych
4. Korelacja wsparta profilerem miedzy ograniczeniami architektury a zachowaniem kernela FEM
5. Natywne kampanie wydajnosciowe FEM na wielu backendach
6. Zagrozenia dla trafnosci i zakres wnioskow

Plan badawczy rozprawy

Teza, pytania badawcze, hipotezy i plan dowodowy.

11. Co trzeba jeszcze dostarczyć przed finalnym pisaniem

Najwazniejsze brakujace elementy naukowe:

- finalna lista platform i backendow,
- zestaw finalnych kampanii,
- statystyka i przedzialy niepewnosci,
- opis profilerow i licznikow,
- rozdzial threats to validity,
- finalny zbior tabel i wykresow.

12. Jednozdaniowa narracja pracy

Najbezpieczniejsza finalna narracja brzmi:

- Rdzeniem pracy jest metodologia, w ktorej mikrobenchmarki architektury sa laczone z realistycznym kernelem FEM przez probe'y walidacyjne, replay poprawnosci na zamrozonych danych oraz korelacje profilerowe.

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

Plik źródłowy: docs/THESIS_NAMING_MAP.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

Ten dokument porządkuje nazewnictwo pod rozprawę i prezentację.

1. Co jest główną osią pracy

Główną osią pracy są mikrobenchmarki, a nie benchmark referencyjny sam w sobie.

Najbezpieczniejsza narracja:

- mikrobenchmarki opisują architekturę
- realistyczny kernel FEM służy jako test aplikacyjny
- replay poprawności potwierdza, że porównywane backendy liczą to samo
- korelacja profilerowa spina warstwę architektoniczną z warstwą aplikacyjną

2. Nazwy eksperymentów w repo i w rozprawie

Repo / tooling

- reference_exact
- correctness_replay
- native_performance_campaign
- fem_option_validation
- profiler_correlation

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

Rozprawa / tekst naukowy

Rekomendowane nazwy:

- Referencyjny przebieg OpenCL exact
- Metal replay poprawności
- natywna kampania wydajnościowa FEM
- Walidacja opcji FEM probes
- korelacja profilerowa report

3. Czego unikać

Nie polecam nazywać warstwy walidacyjnej:

- Filip microbench
- Filip option microbench
- microbenchmark Filipa

To zaciera granice między:

- Twoim wkładem metodologicznym
- a benchmarkiem referencyjnym, który już istniał

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

4. Jak opisywać benchmark referencyjny

Bezpieczny opis:

- legacy FEM reference kernel campaign
- referencyjny exact campaign based on the original OpenCL implementation
- reference application-level validation kernel

Jesli chcesz wskazac pochodzenie, ale nie oddawac nazwy calemu rozdzialowi, uzywaj raczej:

- reference campaign derived from the original implementation

zamiast robic z tego marke calej warstwy badawczej.

5. Jak opisywać Twoja warstwę walidacyjną

Rekomendowany termin:

- Walidacja opcji FEM probes

Co ten termin komunikuje:

- to jest warstwa probe'ow
- sluzy do walidacji i interpretacji

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

- dotyczy opcji istotnych dla realistycznego kernela FEM
- nie rosi sobie prawa do bycia oryginalnym benchmarkiem referencyjnym

6. Jak opisywać replay

Rekomendowane terminy:

- replay na zamrożonych danych wejściowych poprawności
- cross-backend replay poprawności
- Metal replay on frozen OpenCL inputs

To wzmacnia najważniejszą cechę metodologiczną:

- porównanie backendów dotyczy tego samego obliczenia na tych samych danych wejściowych

7. Jak opisywać korelację profilerowa

Rekomendowane terminy:

- microbenchmark-to-application correlation
- warstwa korelacji wsparta profilerem
- cross-layer interpretation przepływu pracy

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

To dobrze ustawia role tej warstwy:

- nie jako osobnego benchmarku
- tylko jako narzędzia interpretacji i wyjaśniania wyników

8. Propozycja nazw rozdziałów

1. Charakterystyka architektury z użyciem mikrobenchmarków
2. Od mikrobenchmarków do prób walidacyjnych specyficznych dla FEM
3. Referencyjna kampania exact i replay poprawności na zamrożonych danych wejściowych
4. Korelacja wsparta profilerem między ograniczeniami architektury a zachowaniem kernela FEM
5. Natywne kampanie wydajnościowe FEM na wielu backendach

9. Propozycja nazw tabel i wykresów

Tabele

- Podsumowanie mikrobenchmarków w podziale na backendy
- Podsumowanie prób walidacyjnych opcji FEM
- Najlepsze konfiguracje referencyjnego exact
- Podsumowanie błędów replayu poprawności
- Podsumowanie korelacji profilerowej

Mapa nazewnictwa do rozprawy

Spójne słownictwo dla tekstu, wykresów i prezentacji.

Wykresy

- Roofline backendu wyznaczony z maksimów mikrobenchmarków
- Dłty walidacji opcji FEM w podziale na backendy
- Krajobraz opcji referencyjnego exact
- Rozkład błędów replayu poprawności
- Korelacja między deltami prób a najlepsza konfiguracja exact

10. Najsilniejsze zdanie pod doktorat

Najmocniejsza wersja narracji brzmi mniej więcej tak:

- Głównym wkładem jest metodologia łącząca mikrobenchmarki architektury z realistycznym kernelami FEM przez próby walidacyjne, replay poprawności na zamrożonych danych wejściowych oraz korelacje profilery.

To ustawia benchmark referencyjny na jego właściwym miejscu:

- jako referencje i walidator
- a nie jako główny wkład sam w sobie.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

Plik źródłowy: docs/EXPERIMENTAL_PROTOCOL.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

Ten dokument definiuje protokół eksperymentalny dla v3. Jego celem jest:

- zapewnienie porównywalności wyników,
- ograniczenie dryfu eksperymentalnego,
- przygotowanie danych do publikacji i rozprawy.

1. Zasady ogólne

1.1. Rozdziel correctness od performance

Wszystkie eksperymenty muszą być klasyfikowane jako jedna z warstw:

1. mikrobenchmarki
2. fem_option_validation
3. reference_exact
4. correctness_replay
5. native_performance_campaign
6. profiler_correlation

Nie wolno mieszać ich roli interpretacyjnej.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

1.2. Nie porównuj backendów tylko po czasie

Przed porównaniem wydajności:

- dla reprezentatywnych przypadków należy przeprowadzić replay poprawności,
- albo jasno oznaczyć, że dana kampania nie jest replayem 1:1.

1.3. Wszystko archiwizuj jako artefakt

Każda kampania powinna pozostawić:

- summary.json
- pliki CSV / JSONL / MD
- wykresy finalne
- provenance i summary_hash

2. Projekt eksperymentu

2.1. Zmienne niezależne

Warstwa architektoniczna

- backend

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- architektura sprzetowa
- model urządzenia
- wersja sterownika / runtime
- system operacyjny

Warstwa kernela

- operator
- element_type
- wariant qss/sqs/ssq
- coal_read
- coal_write
- compute_all_shape_fun_der
- workspace_*
- padding
- workgroup_size
- n_qp
- n_elements

2.2. Zmienne zalezne

- elapsed_s

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- throughput_gbps
- gflops
- ns_per_unit
- energia i moc, jesli dostępne
- max_abs_diff
- rms_diff
- records_within_tolerance
- wyniki profilerow

3. Warstwa mikrobenchmarki

3.1. Cel

Zmierzyć:

- peak bandwidth,
- pointer latency,
- peak compute,
- sensitivity na rozmiar i wzorzec pracy.

3.2. Minimalna kampania

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

Na kazdej platformie:

- CPU mikrobenchmarki
- GPU mikrobenchmarki dla aktywnego backendu

3.3. Rekomendowane powtorzenia

- smoke test: 1
- exploratory: 3
- paper-ready: ≥ 5

3.4. Artefakty

- CSV benchmarkow
- roofline
- session summary
- wygenerowane wykresy

4. Warstwa Walidacja opcji FEM

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

4.1. Cel

Sprawdzać, jak backend reaguje na kontrolowane probe'y bliskie opcjom realistycznego kernela FEM.

4.2. Parametry startowe

Dla smoke test:

- repeats = 1
- n_elements = 64
- n_qp = 2
- workgroup_size = 32

Dla kampanii publikacyjnej:

- repeats = 3..5
- n_elements i n_qp zgodne z kampania docelowa
- workgroup_size dobrany do architektury lub utrzymany jako stałe pole porównawcze

4.3. Artefakty

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- fem_option_validation.csv
- records.jsonl
- probe_summary.csv
- category_summary.csv
- fem_option_validation.md
- summary.json

4.4. Kryteria interpretacji

- $\text{mean_delta_ratio} < 1.0$ oznacza, że stan toggled jest korzystniejszy od baseline
- recommended_state musi być interpretowane w kontekście probe'u
- probe'y z kategorii profile nie są pojedynczymi kontrolami i nie należy ich interpretować jak pojedynczych toggle'i

5. Warstwa referencyjny exact

5.1. Cel

Uruchomić referencyjną kampanię legacy FEM jako źródło:

- zamrożone dane wejściowe,

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- oczekiwane wyniki,
- metadata launchu,
- exact timings.

5.2. Srodowisko

- Linux
- OpenCL / Intel OpenCL exact
- przygotowany mod_2022

5.3. Artefakty

- summary.json
- combined_csv
- launch_dumps/
- numerical_outputs/
- replay_inputs_bundle/
- canonical_replay_bundles/

6. Warstwa replay poprawności

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

6.1. Cel

Sprawdzać, czy inny backend liczy to samo na tych samych zamrożone dane wejściowe.

6.2. Minimalny protokół

1. Wygeneruj `reference_exact`
2. Wyeksportuj kompaktowy pakiet replay
3. Uruchom replay na backendzie docelowym
4. Sprawdź:
 - `records_checked`
 - `records_within_tolerance`
 - `records_out_of_tolerance`
 - `worst_max_abs_diff`
 - `worst_rms_diff`

6.3. Kryterium akceptacji

Minimalne:

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- records_out_of_tolerance = 0

Jesli nie jest spelnione:

- replay nie moze byc uznany za potwierdzenie correctness
- trzeba zbadac rozbieznosc numeryczna

7. Warstwa native performance campaign

7.1. Cel

Zbadac zachowanie natywnej implementacji projektu na wielu backendach.

7.2. Ograniczenie interpretacyjne

Wyniki natywnej kampanii:

- sa bardzo cenne wydajnosciowo,
- ale nie stanowią same w sobie dowodu 1:1 correctness wobec legacy exact.

Kazda tabela i wykres z tej warstwy powinny to jasno rozrozniac.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

8. Korelacja profilerowa

8.1. Cel

Scalic:

- microbench peaks,
- probe deltas,
- best exact/native configuration,
- raporty profilerowe.

8.2. Minimalne wejścia

Wymagane:

- katalog optimization / exact-native runu
- katalog fem_option_validation

Opcjonalne:

- eksporty profilerow

8.3. Rekomendowane profilery

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

Intel

- VTune
- Advisor / roofline, jeśli dostępne

Apple

- Xcode GPU tools
- Metal System Trace / GPU capture

NVIDIA

- Nsight Compute
- Nsight Systems

AMD

- rocprof / rocprofv2

8.4. Minimalny plan liczników

Na tyle, na ile pozwala platforma, zbieraj:

- achieved bandwidth

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- achieved FLOPS / EU utilization / occupancy
- stall reasons
- kernel time
- memory transactions / cache behavior

9. Powtorzenia i statystyka

9.1. Minimalne zasady

- smoke test: 1
- exploratory runs: 3
- finalne wyniki do rozprawy: ≥ 5

9.2. Raportowane statystyki

Dla kazdej glownej metryki raportuj:

- mean
- sigma / std
- CV
- liczbe runow

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

Jesli to mozliwe, dodaj:

- 95% confidence interval

9.3. Ranking opcji

Przy porownywaniu opcji:

- raportuj nie tylko najlepszy punkt,
- ale tez stabilnosc rankingu miedzy runami / seriami.

10. Warmup, cache i stabilizacja

Przed kampania:

- wykonaj smoke test,
- odnotuj, czy pierwsze uruchomienie ma dodatkowy narzut kompilacji kernela,
- nie mieszaj warmup runow z wynikami finalnymi.

Dla GPU:

- zaznacz, czy pierwszy run zawiera JIT / kernel compilation overhead.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

11. Zasady archiwizacji

Dla każdej kampanii trzymaj:

- katalog wynikowy bez nadpisywania,
- summary.json,
- finalne CSV,
- finalne Markdown reporty,
- najważniejsze PNG,
- bundle, jeśli dotyczy.

Rekomendowany układ:

- artifacts/microbench/<backend>/...
- artifacts/fem_option_validation/<backend>/...
- artifacts/reference_exact/<backend>/...
- artifacts/correctness_replay/<backend>/...
- artifacts/profiler_correlation/<backend>/...

12. Freeze eksperymentalny

Przed finalnym zbiorem danych:

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- nie zmieniaj kodu eksperymentalnego,
- nie zmieniaj nazw artefaktów,
- zamroz:
 - wersje backendów,
 - wersje sterowników,
 - wersje systemów,
 - ustawienia kampanii.

W praktyce:

- oznacz jeden snapshot v3 jako finalny do danych rozprawowych.

13. Kryteria "paper-ready"

Wynik uznaj za gotowy do rozprawy, jeśli:

- ma summary.json,
- ma summary_hash,
- ma provenance,
- ma stabilne powtórzenia,
- ma sensowną interpretację,
- jest przypisany do jednej klasy eksperymentu,
- nie miesza correctness i performance.

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

14. Co raportować w każdym rozdziale eksperymentalnym

Minimalnie:

- konfiguracja sprzętowa,
- backend,
- przepływ pracy,
- parametry,
- liczba powtórzeń,
- główne metryki,
- najważniejsze ograniczenia interpretacyjne.

15. Gotowa checklista przed finalnymi pomiarami

- [] wybrano finalne platformy
- [] ustalono profile quick/exploratory/paper
- [] sprawdzono exact replay na reprezentatywnych przypadkach
- [] przygotowano profiler export plan
- [] potwierdzono, że wszystkie przepływy pracy zapisują summary.json
- [] potwierdzono, że wszystkie przepływy pracy zapisują artefakty raportowe

Protokół eksperymentalny

Zasady kampanii, powtórzeń, poprawności i archiwizacji.

- [] ustalono katalog archiwizacji finalnych wynikow

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Plik źródłowy: docs/THREATS_TO_VALIDITY.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisanie rozprawy.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Ten dokument porządkuje zagrożenia dla trafności wyników i sposób ich ograniczania.

1. Trafność konstruktów

Pytanie:

- czy mierzymy to, co twierdzimy, że mierzymy?

Zagrożenie 1. Mylenie czasu z poprawnością

Ryzyko:

- podobny czas wykonania nie oznacza automatycznie poprawnego wyniku obliczeń.

Mitigacja:

- replay poprawności na zamrożone dane wejściowe,
- porównanie outputów przez `max_abs_diff` i `rms_diff`.

Zagrożenie 2. Mylenie `native_performance_campaign` z `exact replay`

Ryzyko:

- natywna kampania projektu może być przedstawiana jako ściśle 1:1 wobec `legacy exact`.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Mitigacja:

- jawne rozroznienie klas eksperymentów,
- osobne nazewnictwo i osobne sekcje w rozprawie.

Zagrozenie 3. Mylenie high-level input z rzeczywistym wejściem kernela

Ryzyko:

- samo mesh + problem file może być uznane za identyczne wejście.

Mitigacja:

- frozen runtime buffers:
 - execution_parameters.bin
 - gauss_dat.bin
 - shape_fun_ref.bin
 - el_data_in.bin

2. Trafność wewnętrzna

Pytanie:

- czy obserwowane różnice są skutkiem badanych czynników, a nie ukrytych zakłóceń?

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Zagrożenie 1. JIT / first-run overhead

Ryzyko:

- pierwszy run może zawierać koszt kompilacji kernela.

Mitigacja:

- smoke test / warmup,
- jawne odnotowanie pierwszego runu,
- nie mieszanie warmup z wynikami finalnymi.

Zagrożenie 2. Niekontrolowane obciążenie systemu

Ryzyko:

- inne procesy zaburzają CPU/GPU timings.

Mitigacja:

- odseparowane sesje pomiarowe,
- ograniczenie innych zadań,
- zapisywanie provenance i kontekstu uruchomienia.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Zagrożenie 3. Dynamiczne zarządzanie energią i taktowaniem

Ryzyko:

- turbo, power limit, thermal throttling, DVFS.

Mitigacja:

- notatki o stanie systemu,
- powtorzenia,
- interpretacja z profilerem i energetyka tylko tam, gdzie ma to sens.

Zagrożenie 4. Różnice w sterownikach i runtime

Ryzyko:

- backendy są uruchamiane na różnych wersjach runtime i kompilatorów.

Mitigacja:

- provenance,
- hashowanie,
- zapisywanie wersji systemu i środowiska,
- freeze eksperymentalny przed zbiorem finalnym.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

3. Trafność wnioskowania

Pytanie:

- czy wnioski statystyczne są uzasadnione?

Zagrożenie 1. Za mało powtórzeń

Ryzyko:

- losowy szum jest interpretowany jako efekt architektoniczny.

Mitigacja:

- ≥ 5 powtórzeń dla finalnych wyników,
- raportowanie mean, sigma, CV,
- dodatkowe confidence intervals, jeśli to możliwe.

Zagrożenie 2. Nadmierna interpretacja jednego najlepszego wyniku

Ryzyko:

- najlepsza konfiguracja jest niestabilna, ale przedstawiana jako reprezentatywna.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Mitigacja:

- analiza rankingu,
- raportowanie top-k,
- porównanie stabilności między seriami.

Zagrożenie 3. Zbyt słabe kryterium correctness

Ryzyko:

- tolerancja jest ustawiona zbyt luzno.

Mitigacja:

- jawne raportowanie:
 - records_checked
 - records_within_tolerance
 - records_out_of_tolerance
 - worst_max_abs_diff
 - worst_rms_diff

4. External validity

Pytanie:

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

- na ile wyniki uogólniają się poza testowany zestaw?

Zagrożenie 1. Ograniczona liczba architektur

Ryzyko:

- wnioski będą zbyt mocno zależne od jednej platformy.

Mitigacja:

- przynajmniej:
 - CPU,
 - Apple GPU / Metal,
 - Intel GPU / OpenCL,
- opcjonalnie NVIDIA i AMD.

Zagrożenie 2. Ograniczony zestaw przypadków FEM

Ryzyko:

- wyniki są silne tylko dla test_prism i laplace_prism.

Mitigacja:

- uczciwe ograniczenie zakresu twierdzeń,

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

- rozwój dodatkowych cases w przyszłości.

Zagrożenie 3. Dopasowanie probe'ow do jednego kernela

Ryzyko:

- fem_option_validation zbyt mocno odzwierciedla tylko jeden styl kernela.

Mitigacja:

- jawne opisanie, że probe'y są warstwą interpretacyjną dla tej klasy realistycznego kernela FEM,
- nie przedstawianie ich jako uniwersalnych benchmarków dla całego HPC.

5. Tool validity

Pytanie:

- czy same narzędzia nie wprowadzają błędów?

Zagrożenie 1. Błędy parserów raportów

Ryzyko:

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

- źle sparsowany profiler daje mylne wnioski.

Mitigacja:

- przechowywanie raw exports,
- zachowanie plików źródłowych obok raportów korelacyjnych,
- parsowanie do CSV/JSON bez nadpisywania raw danych.

Zagrożenie 2. Dekoder el_data_out

Ryzyko:

- błędna interpretacja płaskiego bufora.

Mitigacja:

- jawne ograniczenie obecnej wersji dekodera do wspieranych przypadków,
- testy regresyjne na canonical bundles.

Zagrożenie 3. Translator replay

Ryzyko:

- translator OpenCL -> Metal nie jest ogólnym kompilatorem i może mieć ograniczony zakres poprawności.

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

Mitigacja:

- utrzymywanie kanoniczne pakiety replay,
- walidacja tylko w zakresie wspieranych przypadków,
- jawne opisanie ograniczeń.

6. Ograniczenia, które trzeba powiedzieć wprost w rozprawie

Należy jawnie napisać, że:

- native_performance_campaign nie jest strict replayem 1:1,
- replay poprawności ma zakres ograniczony do wspieranych przypadków i bundli,
- korelacja profilerowa jest warstwą interpretacyjną, nie dowodem przyczynowości samym w sobie,
- benchmark referencyjny nie jest głównym wkładem pracy.

7. Najważniejsze działania ograniczające ryzyko

Jeśli trzeba wskazać 5 najważniejszych mitigacji, to są nimi:

1. replay na zamrożonych danych wejściowych poprawności,
2. provenance i summary_hash,

Zagrożenia dla trafności

Ograniczenia metodologiczne i ich kontrola.

3. rozdzielenie correctness od performance,
4. korelacja wsparta profilerem,
5. jawne raportowanie ograniczeń i klasy eksperymentu.

8. Minimalna checklista do sekcji "Threats to Validity"

- [] rozróżniono construct/internal/external/conclusion validity
- [] opisano ograniczenia replayu
- [] opisano ograniczenia native campaigns
- [] opisano ograniczenia profilerów
- [] opisano ograniczenia dekodera outputu
- [] wskazano, jak provenance i bundling ograniczają ryzyko

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

Plik źródłowy: docs/END_TO_END_TESTING_AND_WRITING.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

Ten dokument zbiera minimalny, praktyczny przepływ pracy v3 tak, żeby dało się:

- szybko robić smoke testy,
- wykonywać pełne eksperymenty,
- odkładać artefakty wprost pod rozprawę.

Dokumenty uzupełniające:

- docs/V3_DOCUMENTATION_INDEX.md
- docs/THESIS_RESEARCH_PLAN.md
- docs/EXPERIMENTAL_PROTOCOL.md
- docs/SYSTEM_REFERENCE.md

1. Zasada ogólna

Główna os prcy to:

1. mikrobenchmarki
2. fem_option_validation
3. reference_exact
4. correctness_replay
5. profiler_correlation

Kolejność ma znaczenie:

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

- najpierw opis architektury,
- potem probe'y walidacyjne FEM,
- potem referencyjny kernel,
- potem replay poprawnosci,
- na koncu korelacja i interpretacja.

2. Szybki smoke test

2.1. Walidacja opcji FEM

GUI:

- Workflow: Walidacja opcji FEM
- Backend: wybrany backend testowy
- Repeats: 1
- n_elements: 64
- n_qp: 2
- workgroup_size: 32

Oczekiwany wynik:

- katalog w data/fem_option_validation/...
- pliki:

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

- summary.json
- fem_option_validation.csv
- probe_summary.csv
- category_summary.csv
- fem_option_validation.md

2.2. Korelacja profilerowa

Wymagane:

- jeden katalog optimization/exact
- jeden katalog fem_option_validation

GUI:

- Workflow: Korelacja profilerowa
- wskaż:
 - Optimization dir
 - Walidacja opcji FEM dir

Oczekiwany wynik:

- katalog .../profiler_correlation/
- pliki:

Przepływ pracy od testów do pisanja

Checklisty robocze do testów, raportów i rozprawy.

- summary.json
- profiler_correlation.json
- profiler_correlation.md
- option_alignment.csv
- profile_proximity.csv
- category_summary.csv

3. Pełny przepływ pracy correctness

3.1. Linux OpenCL exact

GUI:

- Workflow: Filip original
- Backend: opencl albo intel
- Filip mode: exact_reference
- Filip case: test_prism albo laplace_prism
- zaznacz:
 - Dump OpenCL launch artifacts
 - Export compact replay inputs bundle
- opcjonalnie do walidacji:
 - Include OpenCL output in pakiet replay

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

- Export kanoniczne pakiety replay

Artefakty:

- launch_dumps/
- replay_inputs_bundle/
- opcjonalnie canonical_replay_bundles/

3.2. macOS Metal replay

GUI:

- Workflow: Filip original
- Backend: metal
- Filip mode: exact_reference
- Replay dump root ustaw na:
 - replay_inputs_bundle
 - albo canonical_replay_bundles/<bundle>

Oczekiwany wynik:

- execution_mode = exact_reference_metal_replay
- validation_summary
- zapisane outputy i ich dekodowanie

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

4. Pełny przepływ pracy interpretacyjny

4.1. Mikrobenchmarki

Uruchom:

- CPU benchmark
- GPU benchmark
- ewentualnie real kernels

Zachowaj:

- roofline
- session summaries
- generated plots

4.2. Walidacja opcji FEM

Uruchom probe'y na tym samym backendzie, na którym chcesz interpretować realny kernel.

Zachowaj:

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

- probe_summary.csv
- category_summary.csv
- fem_option_validation.md

4.3. Exact/native run

Uruchom:

- Filip original

Zachowaj:

- summary.json
- combined_csv
- decoded output, jeśli dotyczy
- pakiet replay, jeśli robisz walidację poprawności

4.4. Korelacja profilerowa

Uruchom:

- Korelacja profilerowa

Zachowaj:

Przepływ pracy od testów do pisanja

Checklisty robocze do testów, raportów i rozprawy.

- profiler_correlation.md
- option_alignment.csv
- profile_proximity.csv

5. Minimalny zestaw artefaktów do rozprawy

Na jeden backend / jedna kampanie warto archiwizowac:

- summary.json dla kazdego glownego przeplywu pracyu
- probe_summary.csv
- category_summary.csv
- profiler_correlation.md
- option_alignment.csv
- profile_proximity.csv
- najwazniejsze PNG zapisane z GUI
- kompaktowy pakiet replay lub canonical pakiet replay

6. Co warto wkladac do tabel i wykresow

Tabele

- microbenchmark summary by backend

Przepływ pracy od testów do pisanja

Checklisty robocze do testów, raportów i rozprawy.

- Walidacja opcji FEM probe summary
- best exact/native configuration
- replay poprawności error summary
- korelacja profilerowa summary

Wykresy

- roofline from microbenchmark peaks
- Walidacja opcji FEM delta ratios
- best exact configuration landscape
- replay poprawności error distribution
- correlation alignment bar chart
- profile proximity chart

7. Co sprawdzać przed każdą serią pomiarów

Checklist:

- backend i device-index są poprawne
- summary_hash jest zapisany
- provenance jest obecne
- output directories są rozdzielone między eksperymentami
- pakiet replay zawiera to, czego potrzebujesz

Przepływ pracy od testów do pisania

Checklisty robocze do testów, raportów i rozprawy.

- w replay poprawności masz:
- records_checked
- records_within_tolerance
- records_out_of_tolerance

8. Co sprawdzać przed pisanem rozdziału

Checklist:

- rozróżniasz reference_exact, correctness_replay, native_performance_campaign
- nie mieszasz natywnej kampanii z 1:1 replayem
- każdy wykres ma odpowiadający mu plik z danymi wejściowymi
- każda tabela ma katalog z artefaktami i summary_hash
- wnioski wydajnościowe są wspierane przez:
 - mikrobenchmark
 - probe walidacyjny
 - korelacja profilerowa

9. Rekomendowany porządek katalogów do archiwizacji

Najbezpieczniej trzymać to tak:

Przepływ pracy od testów do pisanja

Checklisty robocze do testów, raportów i rozprawy.

- artifacts/microbench/<backend>/...
- artifacts/fem_option_validation/<backend>/...
- artifacts/reference_exact/<backend>/...
- artifacts/correctness_replay/<backend>/...
- artifacts/profiler_correlation/<backend>/...

To bardzo ułatwia składanie wykresow i tabel bez wracania do surowych eksperymentow.

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

Plik źródłowy: docs/UBUNTU_FILIP_SETUP.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

Ten dokument opisuje przepływ pracy v3 na Linuxie dla warstwy referencyjnej i walidacyjnej:

1. OpenCL exact reference
2. kompaktowy pakiet replay export
3. kanoniczne pakiety replay
4. Walidacja opcji FEM
5. korelacja profilerowa

Dokument nie zastępuje ogólnego setupu mikrobenchmarków. Koncentruje się na tej części, która spina benchmark Filipa z walidacją i analizą.

1. Wymagania systemowe

Na Ubuntu potrzebujesz:

- działającego runtime OpenCL dla urządzenia referencyjnego
- tcsh lub csh
- oneAPI compiler i MKL, jeśli exact ma budować mod_2022
- python3-tk, jeśli chcesz używać GUI

Dokładny bootstrap zależy od maszyny, ale v3 zakłada, że exact runner sam próbuje przygotować:

- PATH dla icx

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

- LD_LIBRARY_PATH dla MKL
- rozpakowanie brakujących mesh_prism.dmp.zip

2. Exact reference z GUI

Uruchom GUI:

```
```bash
```

```
cd /path/to/v3

if [-f .venv/bin/activate]; then source .venv/bin/activate; fi

python run_autotune_gui.py
```

```
```
```

W zakładce Workflows ustaw:

- Workflow: Filip original
- Backend: opencl albo intel
- Filip mode: exact_reference
- Filip case: test_prism, laplace_prism albo prism_pair

Jesli chcesz przygotowac dane pod replay na Metalu, zaznacz:

- Dump OpenCL launch artifacts

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

- Export compact replay inputs bundle

Jesli chcesz od razu walidacje outputu na Macu, zaznacz dodatkowo:

- Include OpenCL output in pakiet replay

Jesli chcesz zbudowac male zestawy regresyjne, zaznacz tez:

- Export kanoniczne pakiety replay

3. Exact reference z CLI

Pełny run exact:

```
```bash
```

```
python run_przepływ pracy.py \
 --przepływ pracy filip_original \
 --backend opencl \
 --filip-mode exact_reference \
 --filip-case test_prism \
 --filip-dump-launch-artifacts \
 --filip-export-replay-inputs \
 --filip-export-replay-include-expected-output \
 --filip-export-replay-include-expected-output
```

# Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

```
--filip-export-canonical-replay-bundles
```

```
...
```

To wygeneruje:

- wynik exact
- pełne launch\_dumps
- kompaktowy pakiet replay
- kanoniczne pakiety replay

## 4. Gdzie trafiają wyniki

Dokładne wyniki exact trafiają do:

- data/optimization/<timestamp>filip\_originalbackend-openclexact

Najważniejsze artefakty w tym katalogu:

- summary.json
- csv/
- raw/
- numerical\_outputs/
- launch\_dumps/

## Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

- replay\_inputs\_bundle/
- canonical\_replay\_bundles/

### 5. Co znajduje się w replay\_inputs\_bundle

To jest lekki zestaw wejść niezbędnych do replayu na innym backendzie.

Minimalny zestaw:

- launch\_meta.json
- execution\_parameters.bin
- gauss\_dat.bin
- shape\_fun\_ref.bin
- el\_data\_in.bin

Opcjonalnie:

- el\_data\_out.bin

Jeśli bundle zawiera el\_data\_out.bin, replay może zrobić automatyczną walidację OpenCL vs Metal.

### 6. Co znajduje się w canonical\_replay\_bundles

## Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

To małe zestawy regresyjne, przeznaczone do:

- szybkich testów poprawności
- testów CI/manualnych po zmianach w replayu
- zmniejszenia kosztu kopiowania pełnych dumpów

Aktualne presety:

- smoke\_test\_prism\_qss\_opt000
- smoke\_test\_prism\_sqs\_opt017
- smoke\_laplace\_prism\_ssq\_opt062

## 7. Walidacja opcji FEM z GUI

W zakładce Workflows ustaw:

- Workflow: Walidacja opcji FEM
- Backend: docelowy backend
- FEM validation ops: np. laplace,test
- Variants: np. qss,sqs,ssq
- n\_elements, n\_qp, workgroup

To wygeneruje wynik w:

## Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

- data/fem\_option\_validation/<timestamp>fem\_option\_validationbackend-...

Najważniejsze pliki:

- fem\_option\_validation.csv
- records.jsonl
- summary.json

## 8. Korelacja profilerowa z GUI

W zakładce Workflows ustaw:

- Workflow: Korelacja profilerowa
- Correlation dirs -> Optimization: katalog exact/native runu z summary.json
- Walidacja opcji FEM dir: katalog wyniku fem\_option\_validation
- Profiler reports: opcjonalne eksporty .json lub .csv

To utworzy raport w:

- <optimization\_dir>/profiler\_correlation/

Pliki wynikowe:

- profiler\_correlation.json
- profiler\_correlation.md

# Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

## 9. Interpretacja wyników

W v3 rozróżniaj trzy klasy eksperymentów:

- reference\_exact
- referencyjne wykonanie legacy OpenCL
- correctness\_replay
- ten sam frozen input, inny backend, walidacja outputu
- native\_performance\_campaign
- natywny benchmark wydajnościowy projektu

Najważniejsze ograniczenie:

- tylko OpenCL exact -> pakiet replay -> Metal replay pozwala stawiać mocne twierdzenie, że porównujesz to samo obliczenie
- native kampanie są bardzo cenne wydajnościowo, ale nie są dowodem numerycznej równoważności legacy exact

## 10. Najkrótsza ścieżka robocza

# Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

## Linux

### 1. Exact OpenCL z bundle export:

```
```bash
```

```
python run_przepływ pracy.py \  
  --przepływ pracy filip_original \  
  --backend opencl \  
  --filip-mode exact_reference \  
  --filip-case test_prism \  
  --filip-dump-launch-artifacts \  
  --filip-export-replay-inputs \  
  --filip-export-replay-include-expected-output
```

```
```
```

### 2. Walidacja opcji FEM:

```
```bash
```

```
python run_przepływ pracy.py \  
  --przepływ pracy fem_option_validation \  
  --backend opencl \  
  --filip-mode exact_reference
```

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

```
--repeats 3
```

```
...
```

3. Skopiuj tylko `replay_inputs_bundle/` na maszynie z Metalem.

macOS

1. W GUI ustaw:

- Workflow: Filip original
- Backend: metal
- Filip mode: exact_reference
- Replay dump root: wskaz na skopiowany `replay_inputs_bundle`

2. Uruchom replay.

3. Jeśli bundle zawierał expected output, sprawdź w Results sekcje Validation.

11. Co sprawdzać w `summary.json`

Najważniejsze pola v3:

- `experiment_class`

Konfiguracja Linux/OpenCL i exact

Środowisko referencyjne, replay i eksport danych wejściowych.

- mode_label
- numerical_equivalence
- replay_input_bundle
- canonical_replay_bundles
- numerical_outputs
- validation_summary
- provenance
- summary_hash

To jest minimalny zestaw, który warto archiwizować do doktoratu razem z wykresami i tabelami.

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

Plik źródłowy: docs/CSV_SCHEMA.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

CSV schema (legacy canonical base, still used in v3)

Uwaga:

- ten dokument opisuje kanoniczne pola CSV odziedziczone z warstwy v2,
- w v3 pozostaje aktualny jako baza dla metryk i eksportów,
- dokumenty nadrzędne dla v3:
 - docs/V3_DOCUMENTATION_INDEX.md
 - docs/SYSTEM_REFERENCE.md
 - docs/EXPERIMENTAL_PROTOCOL.md

Ten dokument opisuje kanoniczny schemat kolumn w plikach CSV generowanych przez `apple_microbench_variant2_streamfix`. Starsze pola (legacy) mogą dalej istnieć dla kompatybilności, ale analiza i publikacje powinny używać tylko pól kanonicznych.

1. Pola wspólne (CPU / GPU / real_kernels)

- timestamp: ISO 8601, YYYY-MM-DDTHH:MM:SS, lokalny czas hosta.
- system: wartość z `platform.system()` (np. Linux, Darwin, Windows).
- node: `hostname (platform.node())`.
- release: `platform.release()`.
- version: `platform.version()`.

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

- machine / arch: architektura (x86_64, arm64, ...).
- processor / cpu_model: opis modelu CPU.
- python_version: wersja Pythona.
- benchmark: identyfikator benchmarku (np. bandwidth, stream, compute_fma, compute_fma_peak, gpu_bandwidth, gpu_compute_fma, gemm, stencil2d, spmv, ...).
- run_id: numer powtórzenia w obrębie danego zestawu parametrów (0..N-1).
- elapsed_s: czas trwania mierzonej części runu [sekundy].

2. CPU - mikrobenchmarki

2.1 Bandwidth / STREAM

- size_mb: rozmiar danych per-array [MiB] (logiczny, niekoniecznie fizycznie zmapowany w RAM).
- n_elements (opcjonalne): liczba elementów typu float32.
- bytes_per_iter: liczba bajtów przetwarzanych w jednej iteracji pętli.
- iters: liczba iteracji pętli wewnętrznej.
- kernel (STREAM): copy | scale | add | triad.
- threads (dla wariantów *_mt): liczba wątków pracera.
- throughput_gbps: przepustowość pamięci [GB/s], zawsze liczona względem 1e9.

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

Legacy:

- gbps – alias na throughput_gbps (w v2 utrzymywany, ale kanoniczne pole to throughput_gbps).

2.2 Pointer latency

- working_set_kb: rozmiar working setu [KiB].
- latency_ns: średnia latencja jednego kroku pointer-chasing [ns].

2.3 Compute FMA

- vector_len / n: długość wektora.
- iters_inner: liczba iteracji wewnętrznych.
- gflops: osiągnięta wydajność [GFLOP/s].

2.4 Peak FMA

Dodatkowe pola:

- threads / num_threads: liczba wątków.
- n_per_thread: liczba elementów na wątek.
- gflops lub:

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

- gflops_peak
- gflops_mean
- gflops_sigma

3. GPU - mikrobenchmarki

Obowiązkowe pola identyfikacji:

- backend: metal | cuda | hip | opencl | inne.
- gpu_model: opis GPU (np. apple_m2_pro, NVIDIA GeForce RTX 3070 Ti Laptop GPU).
- size_bytes: liczba bajtów przetwarzanych w jednym „bloku” danych.
- iters_inner / inner_iters (jeśli dotyczy).
- transfer_kind (bandwidth): device_to_device | host_to_device | device_to_host.
- memory_mode (opcjonalne): np. pinned, pageable, managed.
- copy_method (opcjonalne): np. memcpy, kernel.

Metryki:

- throughput_gbps: przepustowość [GB/s], liczona względem 1e9.
- latency_ns: pointer-chasing latency [ns] (jeśli dotyczy).
- gflops / throughput_gflops: GFLOP/s.

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

- gflops_peak, gflops_mean, gflops_sigma – dla peak FMA.

Legacy:

- gbps – alias na throughput_gbps.

4. real_kernels

Minimalny wspólny zestaw:

- benchmark: gemm | reduction | stencil2d | stencil3d | spmv | fem | fem_integration.
- backend: cpu | cuda | metal (lub inne).
- problem_size: liczbowy opis wielkości problemu (np. liczba elementów, NNZ, itp.).
- shape (opcjonalne): zapis rozmiarów, np. MxNxK.
- elapsed_s: czas całkowity kernela / pętli benchmarku.
- throughput_gbps lub gflops – w zależności od natury problemu.
- status: ok | not_implemented | error:<...>.

5. Energia i moc

Pola standardowe (jeśli energia jest mierzona):

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

- energy_j: energia pobrana w trakcie runu [J].
- avg_power_w: średnia moc [W] (energia / czas).
- energy_source: źródło pomiaru, np.:
 - rapl
 - powermetrics
 - nvml
 - nvidia_smi
 - sysfs
 - powermetrics_gpu
 - fallback_zero / unsupported_* / no_gpu_energy_backend
- energy_supported: 0 / 1 – czy pomiar energii był realnie dostępny.
- energy_samples: liczba próbek mocy użytych do integracji (0 jeśli tylko fallback).
- energy_confidence: heurystyczny poziom zaufania 0.0–1.0.

Pochodne (opcjonalne, ale rekomendowane):

- j_per_gb: energia na 1 GB przetworzonych danych [J/GB].
- j_per_gflop: energia na 1 GFLOP [J/GFLOP].
- edp: Energy-Delay Product (energia * czas) [J·s].

6. Uwagi dot. zgodności wstecznej (v1 → v2)

Schemat danych CSV

Znaczenie kolumn i spójność artefaktów tabelarycznych.

- Skrypty analityczne w v2 powinny czytać zarówno pola legacy, jak i kanoniczne, ale wszystkie nowe benchmarki powinny już zapisywać tylko schemat kanoniczny.
- Dla istniejących benchmarków, rozszerzanie o nowe kolumny (np. `energy_source`, `energy_supported`, `energy_confidence`) jest preferowane zamiast zmiany nazw istniejących kolumn.

Pełna dokumentacja v3

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

Plik źródłowy: docs/METRICS.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

Standard metryk (v2)

Ten dokument definiuje metryki używane w `apple_microbench_variant2_streamfix/v2`

tak, aby:

- były jednoznaczne (brak wątpliwości co do jednostek),
- dało się je wykorzystać w publikacjach naukowych (doktorat),
- były spójne między CPU / GPU / `real_kernels`.

1. Metryki czasu

- `elapsed_s`
 - Definicja: czas trwania mierzonej sekcji runu (kernel loop, pętla benchmarku).
 - Jednostka: sekundy `[s]`.
 - Uwagi:
 - Dla części benchmarków jest to czas pojedynczego runu (zawiera `iters`),
nie pojedynczego wywołania kernela.
 - W opisach publikacyjnych należy jasno zaznaczyć, czego dotyczy „`elapsed`”.

2. Przepustowość pamięci

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

- throughput_gbps

- Definicja:

$$\text{throughput_gbps} = \frac{\text{bytes_total}}{\text{elapsed_s} \cdot 10^9}$$

- Jednostka: gigabajty na sekundę [GB/s], zawsze (10^9) bajtów.

- bytes_total:

- CPU STREAM: suma bajtów czytanych i zapisywanych zgodnie z regułami STREAM, zliczana przez $\text{bytes_per_iter} \cdot \text{iters}$.

- CPU bandwidth: zwykle $2 \cdot \text{size_bytes} \cdot \text{iters}$ (read + write).

- GPU bandwidth: analogicznie, zależnie od transfer_kind.

- Uwagi:

- Dla porównań CPU vs GPU zawsze należy upewnić się, że liczymy

bytes_total w ten sam sposób (czy liczymy tylko payload, czy również

dodatkowe kopie / buforowanie).

Legacy:

- gbps – alias na throughput_gbps; w nowym kodzie preferujemy nazwę throughput_gbps.

3. Wydajność obliczeniowa

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

- gflops / throughput_gflops

- Definicja:

$\backslash$

$$\text{GFLOP/s} = \frac{\text{flops_total}}{\text{elapsed_s} \cdot 10^9}$$

$\backslash$

- Jednostka: giga operacji zmiennoprzecinkowych na sekundę [GFLOP/s].

- flops_total:

- dla FMA: $(2 \cdot n_{\text{elements}} \cdot \text{iters_inner})$,

- dla GEMM: klasyczne $(2MNK)$ dla mnożenia macierzy $(M \times K)$ i $(K \times N)$.

- gflops_peak, gflops_mean, gflops_sigma

- Używane w benchmarkach typu „peak”, gdzie:

- gflops_peak – maksymalna wartość z wielu runów,

- gflops_mean – średnia arytmetyczna,

- gflops_sigma – odchylenie standardowe (populacyjne).

4. Latencja

- latency_ns

- Definicja: średni czas pojedynczej operacji / kroku (np. pointer-chasing)

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

liczony jako:

$$\text{latency_ns} = \frac{\text{elapsed_s}}{\text{liczba_operacji}} \cdot 10^9$$

- Jednostka: nanosekundy [ns].
- Używana głównie w:
 - CPU pointer-chasing (run_pointer_latency.py),
 - GPU pointer-chasing (Metal/CUDA/HIP/OpenCL).

5. Energia i moc

- energy_j
 - Definicja: energia pobrana w trakcie runu, obliczona jako różnica licznika energii przed i po runie lub jako całka z mocy po czasie.
 - Jednostka: džule [J].

- avg_power_w

- Definicja:

$$\text{avg_power_w} = \frac{\text{energy_j}}{\text{elapsed_s}}$$

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

jeśli obie wartości są dodatnie i skończone.

- Jednostka: waty [W].

- energy_source

- Źródło pomiaru energii/mocy (patrz CSV_SCHEMA.md).

- Krytyczne przy interpretacji wyników – np. rapl vs powermetrics vs nvml.

- energy_supported, energy_samples, energy_confidence

- energy_supported: czy backend realnie mierzył energię (1) czy użył fallbacku (0).

- energy_samples: liczba próbek mocy użytych do integracji (0 → tylko fallback).

- energy_confidence: heurystyczna jakość pomiaru (0-1), pełne znaczenie

opisane w dokumentacji modułu energy_utils.py.

6. Metryki efektywności energetycznej

- j_per_gb

- Definicja:

[

$$\text{j_per_gb} = \frac{\text{energy_j}}{\text{bytes_total} / 10^9}$$

]

- Jednostka: J/GB (energia na gigabajt przetworzonych danych).

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

- Interpretacja:

- Niższa wartość → lepsza efektywność energetyczna transferu pamięci.

- j_per_gflop

- Definicja:

$$\text{j_per_gflop} = \frac{\text{energy_j}}{\text{flops_total} / 10^9}$$

- Jednostka: J/GFLOP.

- Interpretacja:

- Niższa wartość → lepsza efektywność energetyczna obliczeń.

- edp (Energy-Delay Product)

- Definicja:

$$\text{edp} = \text{energy_j} \cdot \text{elapsed_s}$$

- Jednostka: J·s.

- Interpretacja:

- Niższa wartość oznacza lepszy kompromis między energią a czasem wykonania.

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

7. Roofline

- ai_flop_per_byte

- Intensywność obliczeniowa (Arithmetic Intensity):

$$\text{AI}$$

$$\text{AI} = \frac{\text{FLOP}}{\text{bytes}}$$

$$\text{AI}$$

- Jednostka: FLOP/byte.

- peak_bw_gbps, peak_gflops

- Wartości „sufitów” wyznaczane na podstawie pików z mikrobenchmarków

(zob. analysis/roofline_model.py).

- bw_limited_gflops, attainable_gflops, regime

- bw_limited_gflops = AI * peak_bw_gbps.

- attainable_gflops = min(peak_gflops, bw_limited_gflops).

- regime = "memory-bound" jeśli bw_limited_gflops < peak_gflops,
w przeciwnym razie "compute-bound".

8. Wskazówki do publikacji

Definicje metryk

Czas, throughput, GFLOP/s, energia i metryki roofline.

- Zawsze podawaj:
 - jednostki (GB/s, GFLOP/s, ns, J, W),
 - źródło energii (energy_source),
 - liczbę powtórzeń (runs) oraz sposób agregacji (mean $\pm$ sigma, CV).
- Przy porównaniach CPU vs GPU:
 - upewnij się, że bytes_total i flops_total są liczone w ten sam sposób,
 - komentuj, czy benchmark odzwierciedla realne workloady (tu pomaga moduł real_kernels).

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

Plik źródłowy: docs/REPRODUCIBILITY.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

Reproducibility guide (legacy base, adapted in v3)

Uwaga:

- ten dokument pochodzi z warstwy v2 i nadal zawiera przydatne wskazówki ogólne,
- dla aktualnego przepływu pracy v3 najpierw czytaj:
 - docs/V3_DOCUMENTATION_INDEX.md
 - docs/EXPERIMENTAL_PROTOCOL.md
 - docs/END_TO_END_TESTING_AND_WRITING.md

Ten dokument opisuje, jak powtarzalnie uruchamiać benchmarki w `apple_microbench_variant2_streamfix` tak, aby wyniki nadawały się do wykorzystania w pracy doktorskiej.

1. Środowisko i zależności

- Wymagania ogólne:
 - Python 3.10+.
 - Zestaw standardowych bibliotek naukowych (numpy, matplotlib, itp.).
- Dla GPU:
 - CUDA toolkit + sterowniki (dla backendu cuda),
 - ROCm (dla backendu hip),

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

- sterownik z OpenCL (dla backendu opencl),
- macOS 13+ z Metal (dla backendu metal).
- Dodatkowo dla energii:
 - Linux/x86: dostęp do /sys/class/powercap/*rapl*/energy_uj (RAPL).
 - macOS: narzędzie powermetrics (zwykle wymaga sudo, szczególnie dla GPU).
 - NVIDIA: pynvml lub nvidia-smi w PATH.

2. Budowanie bibliotek

CPU:

```
```bash
```

```
cd v2

cd cpu/lib

./build_linux.sh # na Linux

./build_mac.sh # na macOS
```

```
```
```

GPU:

- CUDA:

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

```
```bash
```

```
cd v2

cd gpu/cuda/lib

./build_cuda.sh
```

```
```
```

- HIP:

```
```bash
```

```
cd v2

cd gpu/hip/lib

./build_hip.sh
```

```
```
```

Metal i OpenCL nie wymagają budowania bibliotek natywnych (kod kerneli jest kompilowany przez sterownik w czasie uruchomienia).

3. Podstawowe profile uruchomieniowe

Repozytorium korzysta z profili eksperymentów zdefiniowanych w:

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

- configs/experiment_profiles.json
- configs/platform_profiles.json

Główne profile:

- quick – minimalny zestaw rozmiarów i liczby runów (smoke / sanity).
- paper – domyślny profil do zbierania danych pod publikację.
- full – rozszerzony zestaw (większa rozdzielczość rozmiarów / więcej runów).

Przykład uruchomienia profilu paper:

```
```bash
cd v2
python3 run_all_backends.py --profile paper
```
```

Wygeneruje to katalog sesji w data/runs/ oraz manifest (manifest.json).

4. Zalecenia dot. ustawień CPU i GPU

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

CPU:

- Jeśli to możliwe:
 - ustaw governor na performance (Linux),
 - wyłącz dynamiczne skalowanie częstotliwości (lub przynajmniej zanotuj jego stan),
 - nie uruchamiaj innych ciężkich zadań równolegle.
- W raportach zawsze zapisuj:
 - model CPU,
 - liczbę rdzeni / wątków,
 - informację o turbo / governorze (choćby w formie tekstowej notatki).

GPU:

- Upewnij się, że GPU nie jest współdzielone z innymi ciężkimi zadaniami.
- Dla NVIDIA:
 - jeśli ustawiasz limit mocy, zanotuj go (np. `nvidia-smi -pl 120`).
- W raportach zapisuj:
 - model GPU,
 - wersję sterownika,
 - tryb pracy (domyślny / z limitem mocy).

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

5. Energia i jakość pomiaru

Pomiar energii realizowany jest przez:

- energy.py – głównie CPU (RAPL, powermetrics).
- energy_utils.EnergyLogger – CPU i GPU (NVML, nvidia-smi, sysfs, powermetrics GPU).

Rekomendacje:

- Dla CPU/Linux: uruchamiaj benchmarki jako zwykły użytkownik – RAPL nie wymaga sudo.
- Dla CPU/GPU na macOS: pomiary powermetrics zwykle wymagają sudo. Jeśli nie możesz używać sudo, traktuj energię jako niedostępną (fallback 0 J / 0 W nie powinien być interpretowany jako „prawdziwy” pomiar).
- W analizach:
 - korzystaj z pól energy_source, energy_supported, energy_confidence,
 - filtruj wiersze o niskim energy_confidence lub z energy_source zawierającym fallback / unsupported, jeśli chcesz rysować wykresy efektywności energetycznej.

6. Typowy pipeline „paper”

1. Uruchom kampanię (np. na maszynie z NVIDIA GPU):

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

```
```bash
```

```
cd v2
```

```
python3 run_all_backends.py --profile paper --platform-profile nvidia
```

```
```
```

2. Sprawdź jakość danych:

```
```bash
```

```
cd v2
```

```
python3 analysis/data_quality.py --scope session --strict
```

```
```
```

Jeśli zwróci kod wyjścia 2 – sprawdź komunikaty, popraw błędy (np. dziwne CSV, brakujące pola) i powtórz pomiary.

3. Wygeneruj raport:

```
```bash
```

```
python3 analysis/report.py --mode latest --roofline-target gpu --roofline-backend
--with-plots
```

```
```
```

Odtwarzalność eksperymentów

Provenance, hashe i zamrażanie środowiska eksperymentalnego.

Raport pojawi się w `reports/report_YYYYMMDD_HHMMSS.md`.

4. Zarchiwizuj artefakty:

- katalog sesji w `data/runs/`,
- raport z `reports/`,
- wersje sterowników (np. `output nvidia-smi`, `uname -a`, itp. – jako osobne pliki tekstowe).

7. Minimalne kryteria „paper-ready”

Przed użyciem wyników w publikacji:

- [] `analysis/data_quality.py --scope session --strict` przechodzi bez błędów.
- [] `analysis/gpu_summary.py --mode latest --strict` przechodzi bez błędów.
- [] Dla każdego kluczowego wyniku:
 - liczba runów ≥ 3 ,
 - raportujesz $\text{mean} \pm \text{sigma}$ i współczynnik zmienności (CV),
 - opisujesz źródło pomiaru energii.
- [] Istnieje zapisany manifest sesji oraz notatka o konfiguracji systemu (CPU, GPU, sterowniki).